claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e5140909-890\agent-a7870ad11ffc5cd0d.jsonl`
- SHA-256: `a462ceaee40c33072fe92c686d316b8755c7f8f074496e152e0b41b8174d1975`
- Source modified: `2026-06-18T07:50:07+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_e5140909-890`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T07:47:49.188Z)

```
Work READ-ONLY in this git worktree pinned to origin/master (DO NOT modify):
  C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base
Read the ACTUAL code (and the matching tests/ file) before asserting anything; never guess.
Adversarial bug-hunter. Report ONLY real, reachable defects (crash on a reachable input, wrong
logic, inverted condition, bad error handling, resource leak, off-by-one, contract mismatch,
unguarded external/persisted-data access). Exact file+line + concrete trigger + minimal fix +
confidence. PRECISION over recall ? empty list is a respected answer; do NOT invent findings.
The repo has ~855 passing tests + several hardening sweeps; expect few. Skip pure style.
AREA: backend/eval/ ? focus on the RECENTLY-CHANGED quality code: tolerance_metrics.py,
rally_seg_eval.py, served_gate_a.py, windowing.py, experiment.py (incl. the moved
_is_fusion_schema), per_user_gate.py, promotion.py, regression.py. Numeric edge cases (n=0/1,
div-by-zero, NaN), matching boundaries, schema/IO access.
```

2. assistant (2026-06-18T07:47:51.457Z)

```
I'll start by reading the recently-changed quality code files in backend/eval/ to hunt for real
defects.
```

3. user (2026-06-18T07:47:52.421Z)

```
1       """R2 ? task-faithful rally-boundary eval (tolerance-match + over-segmentation guard).
2
3       The headline rally metric (tIoU-F1 is kept as a SECONDARY diagnostic): a prediction
matches a
4       golden rally iff ``|?start| ? ?_start AND |?end| ? ?_end`` under a **strict 1:1
assignment**, so
5       over-production costs precision directly. Reported as a DECOMPOSITION ? ``P/R/F1@?`` +
start/end
6       boundary-error distributions + an over-seg guard (split/merge counts) + a ?-sweep ?
never one
7       number (a single ? misleads; see the design doc).
8
9       Locked decisions + rationale: ``docs/R2_EVAL_METRIC_DESIGN.md`` (owner-reviewed
2026-06-17):
10      asymmetric ? (``?_start=2.0`` forgives the ~1?1.5 s service window; ``?_end=1.5`` keeps
the
11      rally-end honest ? IAA-calibrated later); augment tIoU now, ablation-gate any eventual
swap.
12
13      Pure + stdlib; **numpy/scipy are OPTIONAL** ? used for the exact Hungarian assignment
when present,
14      else a deterministic greedy 1:1 fallback. Both agree on the temporally-ordered, near-
diagonal
15      eligibility graphs that rally intervals produce, so scores are environment-independent.
```

```
    """
    from __future__ import annotations

    import statistics
    from typing import Any, Dict, List, Optional, Tuple

    Interval = Tuple[float, float]
    #: (pred_idx, gt_idx, dstart_signed, dend_signed) ? signed = pred ? gt (positive = pred
is late).
    Match = Tuple[int, int, float, float]

    #: Provisional ? (the design's locked starting point; fixed by the IAA study later).
Always
    #: report the sweep alongside, since the golden END convention carries >1.5 s of noise.
    TAU_START: float = 2.0
    TAU_END: float = 1.5
    SWEEP_TAUS: Tuple[float, ...] = (1.5, 2.0, 2.5, 3.0)


    def _eligible(p: Interval, g: Interval, tau_start: float, tau_end: float) -> bool:
        return abs(p[0] - g[0]) <= tau_start and abs(p[1] - g[1]) <= tau_end


    def _greedy_match_1to1(preds: List[Interval], gts: List[Interval],
                           tau_start: float, tau_end: float) -> List[Match]:
        """Deterministic greedy 1:1: eligible (pred, gt) pairs in ascending combined
boundary error,
        assigned first-come and skipping used indices. Optimal-or-near on the near-diagonal
eligibility
        graph rally intervals produce. The numpy/scipy-free path (and the CI path)."""
        pairs: List[Tuple[float, float, int, int]] = []
        for i, p in enumerate(preds):
            for j, g in enumerate(gts):
                if _eligible(p, g, tau_start, tau_end):
                    # sort key: combined error, then (i, j) for a fully deterministic
tiebreak
                    pairs.append((abs(p[0] - g[0]) + abs(p[1] - g[1]), float(i * 1e-6 + j *
1e-9), i, j))
        pairs.sort()
        used_p: set = set()
        used_g: set = set()
        matches: List[Match] = []
        for _, _, i, j in pairs:
            if i in used_p or j in used_g:
                continue
            used_p.add(i)
            used_g.add(j)
            matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
        matches.sort(key=lambda mm: mm[1])
        return matches


    def _hungarian_match_1to1(preds: List[Interval], gts: List[Interval],
                              tau_start: float, tau_end: float) -> Optional[List[Match]]:
        """Exact max-cardinality (then min total boundary error) via scipy. Returns ``None``
when
        numpy/scipy are unavailable so the caller falls back to greedy."""
        try:
            import numpy as np
            from scipy.optimize import linear_sum_assignment
        except Exception:
            return None
        n, m = len(preds), len(gts)
        size = max(n, m)
        big = 1.0e6
```

```
        cost = np.full((size, size), big, dtype=float)
        for i, p in enumerate(preds):
            for j, g in enumerate(gts):
                if _eligible(p, g, tau_start, tau_end):
                    cost[i, j] = abs(p[0] - g[0]) + abs(p[1] - g[1])
        rows, cols = linear_sum_assignment(cost)
        matches: List[Match] = []
        for i, j in zip(rows.tolist(), cols.tolist()):
            if i < n and j < m and cost[i, j] < big:  # drop forced ineligible assignments
                matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
        matches.sort(key=lambda mm: mm[1])
        return matches


    def match_1to1(preds: List[Interval], gts: List[Interval],
                   tau_start: float = TAU_START, tau_end: float = TAU_END
                   ) -> Tuple[List[Match], float, float, float]:
        """Strict 1:1 tolerance match ? ``(matches, precision, recall, f1)``. ``P = matched
/ n_pred``
        (over-production costs precision), ``R = matched / n_gt``. Exact Hungarian when
scipy is
        present; deterministic greedy otherwise."""
        n, m = len(preds), len(gts)
        if n == 0 or m == 0:
            return [], 0.0, 0.0, 0.0
        matches = _hungarian_match_1to1(preds, gts, tau_start, tau_end)
        if matches is None:
            matches = _greedy_match_1to1(preds, gts, tau_start, tau_end)
        k = len(matches)
        precision = k / n
        recall = k / m
        f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) else
0.0
        return matches, precision, recall, f1


    def _percentile(xs: List[float], p: float) -> float:
        """Linear-interpolated percentile (pure stdlib)."""
        if not xs:
            return 0.0
        s = sorted(xs)
        k = (len(s) - 1) * p / 100.0
        lo = int(k)
        hi = min(lo + 1, len(s) - 1)
        return s[lo] + (s[hi] - s[lo]) * (k - lo)


    def _overlap(a: Interval, b: Interval) -> float:
        return max(0.0, min(a[1], b[1]) - max(a[0], b[0]))


    def boundary_error_report(preds: List[Interval], gts: List[Interval]) -> Dict[str,
Dict[str, float]]:
        """Median + IQR of signed AND absolute ?start, ?end, split start/end ? the "start
solved,
        end open" diagnostic that aims R4.

        Computed over **best-overlap** matches (each GT ? its maximally-overlapping
prediction, any
        positive overlap), NOT the within-? 1:1 matches: a ?-gated set is survivorship-
biased (loose
        ends fail the gate and vanish from the distribution), which would HIDE the very end-
deficit
        this diagnostic exists to surface. Unconditional best-overlap is what the day-1
validation used
```

```
        to find |?end| ? |?start|. (The within-? deficit shows up instead as low
``tol_recall``.)"""
        def dist(vals: List[float]) -> Dict[str, float]:
            av = [abs(v) for v in vals]
            return {
                "signed_median": statistics.median(vals) if vals else 0.0,
                "abs_median": statistics.median(av) if av else 0.0,
                "abs_p25": _percentile(av, 25.0),
                "abs_p75": _percentile(av, 75.0),
                "n": float(len(vals)),
            }
        ds: List[float] = []
        de: List[float] = []
        for g in gts:
            best = max(preds, key=lambda p: _overlap(p, g), default=None)
            if best is not None and _overlap(best, g) > 0.0:
                ds.append(best[0] - g[0])
                de.append(best[1] - g[1])
        return {"dstart": dist(ds), "dend": dist(de)}


    def over_seg_report(preds: List[Interval], gts: List[Interval]) -> Dict[str, int]:
        """The over-segmentation GUARD (per-video no-regression in promotion):
``split_count`` (?2
        preds covering one rally) + ``merge_count`` (one pred ? ?2 rallies). Reuses the
bipartite
        overlap-graph in ``segmentation_metrics.merge_split_report``."""
        from backend.eval.segmentation_metrics import merge_split_report
        r = merge_split_report(preds, gts)
        return {"split_count": int(r["splits"]), "merge_count": int(r["merges"])}


    # ---------------------------------------------------------------------- #
    # R1 ? net over-segmentation promotion guard
    # ---------------------------------------------------------------------- #
    #: Default over-seg cost weights. A MERGE hides a rally (?2 golden rallies collapse into
one
    #: predicted window ? the user can't reach the hidden one ? recall loss at rally
granularity); a
    #: SPLIT only fragments one rally (it is still found, just in pieces). So a merge is the
strictly
    #: worse fault and is weighted higher. (Equal weights also pass the milestone ? see the
R1 evidence;
    #: the asymmetry is the principled default, not a number tuned to force a verdict.)
    WEIGHT_MERGE: float = 2.0
    WEIGHT_SPLIT: float = 1.0


    def over_seg_guard(base: List[Dict[str, Any]], cand: List[Dict[str, Any]], *,
                       weight_merge: float = WEIGHT_MERGE, weight_split: float =
WEIGHT_SPLIT,
                       use_rates: bool = True, eps: float = 1e-9) -> Dict[str, Any]:
        """**Net** over-segmentation promotion guard (the R1 refinement of the strict per-
video rule).

        ``base``/``cand`` are aligned per-video over-seg dicts (the rows ``rally_seg_eval``
already
        emits): each needs ``split_count``/``merge_count`` and ? when ``use_rates``
(default) ?
        ``merge_rate``/``split_rate``. Optional ``name`` keys align the two lists by video
(else by
        position). Returns the promote-or-block verdict for promoting ``cand`` over
``base``.

        **Why the strict rule needed refining.** R2 §2.3.3's guard blocks promotion if
```

```
``cand`` raises
182         ``split_count`` OR ``merge_count`` on ANY video. That is right for an *incremental*
cue (same
183         windowing structure, an increase = a real regression) but mis-fires on a
*structural* change
184         (mega-windows ? bounded): splitting a single mega-window necessarily lifts
``split_count`` from
185         ~0, and the raw merge **count** is non-monotonic ? one mega-window swallowing 40
rallies is
186         ``merge_count=1`` yet ``merge_rate=1.0`` (every rally hidden), whereas bounding it
into pieces
187         that each glue 2 neighbours is ``merge_count=9`` yet ``merge_rate=0.5`` (FEWER
rallies hidden).
188         So the strict count rule blocks a config that strictly *reduces* the fraction of
rallies lost to
189         over-merge. (Measured: the cap6+R4 milestone trips strict on 10/11 videos yet cuts
mean
190         ``merge_rate`` 0.694 ? 0.150 with NO per-video merge_rate regression ?
RALLY_QUALITY_RESEARCH §6.)
191
192         **The net rule (default verdict ``passes``):** score over-seg as ONE weighted cost
per video and
193         require BOTH:
194           1. the corpus-mean net cost does not rise (``cand_net ? base_net + eps``), AND
195           2. NO video's **merge_rate** rises beyond ``eps`` ? reintroducing a mega-window
that hides MORE
196              of a video's rallies is the one over-merge regression that is never acceptable,
the honest
197              "no NEW merges" rule expressed on the *rate* (the recall-meaningful quantity),
not the count.
198         With ``use_rates`` the per-video cost uses ``merge_rate``/``split_rate`` (fraction
of GT rallies
199         affected, ? [0,1] ? comparable across a structural change); ``use_rates=False``
scores raw counts.
200
201         **Revert (one line):** read ``strict_passes`` instead of ``passes`` to fall back to
the original
202         strict per-video count rule. Both verdicts are always reported, so the choice is
auditable.
203         """
204         def _aligned() -> List[Tuple[Dict[str, Any], Dict[str, Any]]]:
205             if base and cand and all("name" in b for b in base) and all("name" in c for c in
cand):
206                 cmap = {c["name"]: c for c in cand}
207                 return [(b, cmap[b["name"]]) for b in base if b["name"] in cmap]
208             n = min(len(base), len(cand))
209             return [(base[i], cand[i]) for i in range(n)]
210
211         def _cost(row: Dict[str, Any]) -> float:
212             if use_rates:
213                 return weight_merge * float(row["merge_rate"]) + weight_split *
float(row["split_rate"])
214             return weight_merge * float(row["merge_count"]) + weight_split *
float(row["split_count"])
215
216         pairs = _aligned()
217         n = len(pairs)
218         base_costs = [_cost(b) for b, _ in pairs]
219         cand_costs = [_cost(c) for _, c in pairs]
220         base_net = (sum(base_costs) / n) if n else 0.0
221         cand_net = (sum(cand_costs) / n) if n else 0.0
222         net_delta = cand_net - base_net
223
224         # Per-video regressions, reported for transparency.
225         new_merges: List[Dict[str, Any]] = []           # raw COUNT increases (what strict
```

```
trips on)
226         new_splits: List[Dict[str, Any]] = []           # raw COUNT increases (what strict
trips on)
227         merge_rate_regress: List[Dict[str, Any]] = []    # RATE increases (the real over-
merge regression)
228         for b, c in pairs:
229             if float(c["merge_count"]) > float(b["merge_count"]) + eps:
230                 new_merges.append({"name": b.get("name", ""),
231                                    "base": float(b["merge_count"]), "cand":
float(c["merge_count"])})
232             if float(c["split_count"]) > float(b["split_count"]) + eps:
233                 new_splits.append({"name": b.get("name", ""),
234                                    "base": float(b["split_count"]), "cand":
float(c["split_count"])})
235             if use_rates and float(c["merge_rate"]) > float(b["merge_rate"]) + eps:
236                 merge_rate_regress.append({"name": b.get("name", ""),
237                                            "base": float(b["merge_rate"]), "cand":
float(c["merge_rate"])})
238
239         strict_passes = (not new_merges) and (not new_splits)
240         net_cost_ok = net_delta <= eps
241         no_merge_rate_regress = not merge_rate_regress
242         passes = bool(net_cost_ok and no_merge_rate_regress)
243         return {
244             "passes": passes,
245             "strict_passes": strict_passes,
246             "net_cost_ok": net_cost_ok,
247             "no_merge_rate_regress": no_merge_rate_regress,
248             "base_net": base_net,
249             "cand_net": cand_net,
250             "net_delta": net_delta,
251             "n": n,
252             "use_rates": use_rates,
253             "weight_merge": weight_merge,
254             "weight_split": weight_split,
255             "new_merges": new_merges,
256             "new_splits": new_splits,
257             "merge_rate_regress": merge_rate_regress,
258         }
259
260
261     def tolerance_report(preds: List[Interval], gts: List[Interval],
262                          tau_start: float = TAU_START, tau_end: float = TAU_END) ->
Dict[str, object]:
263         """The full R2 block for one video: P/R/F1@? + boundary-error distributions + over-
seg guard."""
264         matches, precision, recall, f1 = match_1to1(preds, gts, tau_start, tau_end)
265         return {
266             "tau_start": tau_start, "tau_end": tau_end,
267             "n_pred": len(preds), "n_gt": len(gts), "matched": len(matches),
268             "precision": precision, "recall": recall, "f1": f1,
269             "boundary_error": boundary_error_report(preds, gts),
270             "over_seg": over_seg_report(preds, gts),
271         }
272
273
274     def tolerance_sweep(preds: List[Interval], gts: List[Interval],
275                         taus: Tuple[float, ...] = SWEEP_TAUS) -> Dict[float, float]:
276         """Symmetric-? F1 sweep ? the trend curve (a single ? misleads). Monotonic non-
decreasing in ?."""
277         return {t: match_1to1(preds, gts, t, t)[3] for t in taus}
278
```

4. user (2026-06-18T07:47:52.820Z)

```
"""One-command rally-segmentation eval ? score a windowing against the golden labels with the
    over-segmentation-sensitive metric set, honestly.

    This is the harness the rally-quality project (``docs/RALLY_QUALITY_RESEARCH.md`` §4.5 / §7)
    measures every tier against. It closes the loop the over-merge finding exposed: plain temporal
    IoU-F1 is blind to merged mega-windows, so we report it **alongside** segmental F1@k, the
    segment-count ratio, and the explicit ``merge_split`` breakdown (``segmentation_metrics``).

    For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds candidate
    windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the SERVED
    operating point and any future windowing the same way) ? scores the windows-as-rallies against
    the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI, and can
    diff two windowings with a paired sign-test + CI (the per-tier ``delta``).

    GROUND TRUTH: the golden manifest (``output/human_lovo_manifest.json``) gives each video's
    trajectory path + fps + frame_width; the GT rally intervals come from
    ``output/<name>_golden_features.json`` (the ``gts`` key) so this needs no extra label files.

    Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
    """
    from __future__ import annotations

    import argparse
    import json
    import os
    from typing import Any, Dict, List, Optional

    from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
    from backend.eval.metrics import Interval
    from backend.eval.windowing import PRESETS, SERVED, WindowingPreset

    DEFAULT_MANIFEST = os.path.join("output", "human_lovo_manifest.json")
    DEFAULT_FEATURES_DIR = "output"
    DEFAULT_IOU = 0.5


    # ---------------------------------------------------------------------------- #
    # Pure scoring core (no I/O ? unit-testable)
    # ---------------------------------------------------------------------------- #
    def score_intervals(preds: List[Interval], gts: List[Interval],
                        iou: float = DEFAULT_IOU,
                        tau_start: Optional[float] = None,
                        tau_end: Optional[float] = None) -> Dict[str, Any]:
        """The full per-video metric row for predicted vs ground-truth rally intervals.

        Pairs temporal-IoU P/R/F1 + boundary error (``metrics.score``) with the
        over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown) so a
        merged mega-window is *visible* even when IoU-F1 isn't moved.

        Also carries the **R2** task-faithful block (``tolerance_metrics``, the headline going forward;
        tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision): strict-1:1
```

```
52          tolerance-match ``tol_f1/tol_precision/tol_recall`` (over-production costs
precision) + the
53          start/end boundary-error medians + the over-seg guard counts. See
docs/R2_EVAL_METRIC_DESIGN.md.
54          """
55          from backend.eval import tolerance_metrics as tm
56          ts = tm.TAU_START if tau_start is None else tau_start
57          te = tm.TAU_END if tau_end is None else tau_end
58          sc = metrics.score(preds, gts, iou_threshold=iou)
59          f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
60          ms = segmentation_metrics.merge_split_report(preds, gts)
61          tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
62          be = tm.boundary_error_report(preds, gts)  # unconditional best-overlap (the
R4-aiming diagnostic)
63          osr = tm.over_seg_report(preds, gts)
64          return {
65              "f1": sc.f1, "precision": sc.precision, "recall": sc.recall, "mean_iou":
sc.mean_iou,
66              "mean_start_error": sc.mean_start_error, "mean_end_error": sc.mean_end_error,
67              "f1@0.1": f1k[0.1], "f1@0.25": f1k[0.25], "f1@0.5": f1k[0.5],
68              "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
69              "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
70              "merges": ms["merges"], "splits": ms["splits"], "missed": ms["missed"],
71              "spurious": ms["spurious"], "num_pred": len(preds), "num_gt": len(gts),
72              # --- R2 (tolerance-match) ? the task-faithful headline block ---
73              "tau_start": ts, "tau_end": te,
74              "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
75              "tol_matched": float(len(tol_matches)),
76              "dstart_abs_median": be["dstart"]["abs_median"], "dend_abs_median":
be["dend"]["abs_median"],
77              "split_count": osr["split_count"], "merge_count": osr["merge_count"],
78          }
79
80
81      _AGG_KEYS = ("f1", "f1@0.25", "f1@0.5", "merge_rate", "split_rate",
"segment_count_ratio",
82                   "precision", "recall",
83                   "tol_f1", "tol_precision", "tol_recall", "dstart_abs_median",
"dend_abs_median")
84
85
86      def aggregate(rows: List[Dict[str, Any]]) -> Dict[str, Any]:
87          """Aggregate per-video rows: mean (+ bootstrap 95% CI) of the headline metrics
across videos.
88
89          Per-video means (not pooled) keep videos the unit of evidence, matching the
harness's
90          leave-one-video-out discipline (windows within a video are correlated)."""
91          out: Dict[str, Any] = {"n": len(rows)}
92          for k in _AGG_KEYS:
93              vals = [r[k] for r in rows]
94              out[k] = eval_stats.mean_with_ci(vals) if vals else None
95          return out
96
97
98      # ------------------------------------------------------------------------ #
99      # Golden corpus I/O
100     # ------------------------------------------------------------------------ #
101     def load_golden(manifest_path: str = DEFAULT_MANIFEST,
102                     features_dir: str = DEFAULT_FEATURES_DIR) -> List[Dict[str, Any]]:
103         """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
104         intervals (``<name>_golden_features.json`` ``gts``). Videos with no GTs are kept but
flagged."""
105         with open(manifest_path) as fh:
```

```
            manifest = json.load(fh)
        out: List[Dict[str, Any]] = []
        for v in manifest:
            feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
            gts: List[Interval] = []
            if os.path.exists(feat):
                with open(feat) as fh:
                    gts = [(float(s), float(e)) for s, e in (json.load(fh).get("gts") or
[])]
            out.append({"name": v["name"], "trajectory": v["trajectory"],
                        "fps": float(v["fps"]), "frame_width": float(v["frame_width"]),
"gts": gts})
        return out


    def windows_for(video: Dict[str, Any], preset: WindowingPreset,
                    min_crossings: int = 0) -> List[Interval]:
        """Parse a video's trajectory, window it under ``preset`` ? predicted rally
intervals.

        ``min_crossings`` > 0 applies the **net-crossings rally-state gate** (Gate-A,
        ``rally_gate.gate_windows``): drop windows whose shuttle crosses the net fewer than
this many
        times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only,
derived
        from the trajectory + an auto-estimated net axis; it cleans up the over-split that
bounded
        windowing (W1) leaves behind.

        NEVER-EMPTY safeguard (mirrors ``FusionSegmenter._select_for_handoff``): if the gate
would
        drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate
is
        strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no
harm".
        """
        from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
        points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
        wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
        ivs = windowing.windows_to_intervals(wins)
        if min_crossings > 0 and ivs:
            from backend.pipeline.detectors.rally_gate import gate_windows
            gated = [(g.start, g.end)
                     for g in gate_windows(points, ivs, video["fps"],
min_crossings=min_crossings)
                     if g.kept]
            # Gating emptied a non-empty video ? keep the ungated windows.
            ivs = gated if gated else ivs
        return ivs


    def evaluate(golden: List[Dict[str, Any]], preset: WindowingPreset,
                 iou: float = DEFAULT_IOU, min_crossings: int = 0,
                 tau_start: Optional[float] = None, tau_end: Optional[float] = None) ->
Dict[str, Any]:
        """Score every golden video under ``preset`` (+ optional net-crossings gate); return
per-video
        rows + the aggregate. Each row carries both the tIoU set and the R2 tolerance-match
block."""
        rows: List[Dict[str, Any]] = []
        for v in golden:
            if not v["gts"] or not os.path.exists(v["trajectory"]):
                continue
            preds = windows_for(v, preset, min_crossings)
            row = score_intervals(preds, v["gts"], iou, tau_start, tau_end)
```

```
158             row["name"] = v["name"]
159             rows.append(row)
160         return {"preset": preset.to_dict(), "iou": iou, "min_crossings": min_crossings,
161                 "rows": rows, "aggregate": aggregate(rows)}
162
163
164     def compare(golden: List[Dict[str, Any]], base: WindowingPreset, cand: WindowingPreset,
165                 iou: float = DEFAULT_IOU, base_min_crossings: int = 0,
166                 cand_min_crossings: int = 0,
167                 tau_start: Optional[float] = None, tau_end: Optional[float] = None,
168                 guard_weight_merge: Optional[float] = None,
169                 guard_weight_split: Optional[float] = None,
170                 guard_use_rates: bool = True) -> Dict[str, Any]:
171         """Baseline vs candidate (windowing and/or net-crossings gate): per-video paired ?
(+ CI +
172         sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the per-tier 'number'. Also
reports ? on the
173         over-segmentation metrics AND the **R1 net over-seg promotion guard**
(``over_seg_guard``):
174         the honest promote/block verdict for cand-over-base, with the strict per-video rule
reported
175         alongside so the refinement is auditable."""
176         from backend.eval import tolerance_metrics as tm
177         b = evaluate(golden, base, iou, base_min_crossings, tau_start, tau_end)
178         c = evaluate(golden, cand, iou, cand_min_crossings, tau_start, tau_end)
179         by_name_b = {r["name"]: r for r in b["rows"]}
180         paired = [(by_name_b[r["name"]], r) for r in c["rows"] if r["name"] in by_name_b]
181         delta: Dict[str, Any] = {}
182         for k in ("f1", "f1@0.25", "tol_f1", "merge_rate", "split_rate",
"segment_count_ratio"):
183             delta[k] = eval_stats.paired_delta_ci([rb[k] for rb, _ in paired],
184                                                   [rc[k] for _, rc in paired])
185         gkw: Dict[str, Any] = {"use_rates": guard_use_rates}
186         if guard_weight_merge is not None:
187             gkw["weight_merge"] = guard_weight_merge
188         if guard_weight_split is not None:
189             gkw["weight_split"] = guard_weight_split
190         guard = tm.over_seg_guard([rb for rb, _ in paired], [rc for _, rc in paired], **gkw)
191         return {"base": b, "cand": c, "n_paired": len(paired), "delta": delta,
"over_seg_guard": guard}
192
193
194     # ------------------------------------------------------------------------ #
195     # Reporting
196     # ------------------------------------------------------------------------ #
197     def _ci(m: Optional[Dict[str, Any]]) -> str:
198         return "?" if not m else f"{m['mean']:.3f} [{m['ci_low']:.3f},{m['ci_high']:.3f}]"
199
200
201     def _format_guard(g: Dict[str, Any]) -> str:
202         """Render the R1 net over-seg promotion-guard verdict (with the strict rule for
contrast)."""
203         basis = "rate" if g["use_rates"] else "count"
204         lines = [f"\n## R1 over-seg promotion guard ({basis}-based, "
205                  f"w_merge={g['weight_merge']}, w_split={g['weight_split']})",
206                  f"  NET verdict: {'PASS' if g['passes'] else 'BLOCK'}  "
207                  f"(net cost {g['base_net']:.3f} ? {g['cand_net']:.3f}, ?
{g['net_delta']:+.3f}; "
208                  f"no_merge_rate_regress={g['no_merge_rate_regress']})",
209                  f"  strict per-video rule (R2 §2.3.3, for contrast): "
210                  f"{'pass' if g['strict_passes'] else 'BLOCK'}  "
211                  f"(count increases ? merges:{len(g['new_merges'])}
splits:{len(g['new_splits'])})"]
212         if g["merge_rate_regress"]:
213             names = ", ".join(f"{r['name']}({r['base']:.2f}?{r['cand']:.2f})"
```

```
                              for r in g["merge_rate_regress"])
            lines.append(f"  ? merge_rate REGRESSED on: {names}")
        return "\n".join(lines)


    def format_report(result: Dict[str, Any]) -> str:
        p = result["preset"]
        bounded = []
        if p.get("max_window_duration"):
            bounded.append(f"cap={p['max_window_duration']}" + ("(hard)" if
p.get("hard_cap") else ""))
        if p.get("inactivity_end"):

bounded.append(f"inact_end={p['inactivity_end']}/rt={p['inactivity_rally_thresh']}"
                           f"/md={p['inactivity_min_density']}")
        if p.get("blob_fallback"):
            bounded.append("blob_fallback")
        extra = (", " + ", ".join(bounded)) if bounded else ""
        lines = [f"# Rally-segmentation eval ? windowing '{p['name']}' "
                 f"(vt={p['velocity_thresh']}, merge_gap={p['merge_gap']}, "
                 f"min_win={p['min_window_duration']}{extra}), IoU={result['iou']}", ""]
        lines.append("| video | F1 | F1@0.25 | F1@0.5 | merges | merge_rate | seg_ratio |
pred/gt |")
        lines.append("|---|---|---|---|---|---|---|---|")
        for r in result["rows"]:
            lines.append(f"| {r['name']} | {r['f1']:.3f} | {r['f1@0.25']:.3f} |
{r['f1@0.5']:.3f} | "
                         f"{int(r['merges'])} | {r['merge_rate']:.2f} |
{r['segment_count_ratio']:.2f} | "
                         f"{r['num_pred']}/{r['num_gt']} |")
        a = result["aggregate"]
        lines += ["", f"**Aggregate (n={a['n']}, mean [95% CI]):** "
                  f"F1 {_ci(a['f1'])} · F1@0.25 {_ci(a['f1@0.25'])} · F1@0.5
{_ci(a['f1@0.5'])} · "
                  f"merge_rate {_ci(a['merge_rate'])} · seg_ratio
{_ci(a['segment_count_ratio'])}"]
        # --- R2 (tolerance-match) block ? the task-faithful headline (tIoU above is
secondary) ---
        rows = result["rows"]
        if rows and "tol_f1" in rows[0]:
            ts, te = rows[0]["tau_start"], rows[0]["tau_end"]
            lines += ["", f"## R2 ? tolerance-match (?_start={ts}s, ?_end={te}s, strict
1:1)", "",
                      "| video | tolF1 | tolP | tolR | |?start| | |?end| | split | merge |",
                      "|---|---|---|---|---|---|---|---|"]
            for r in rows:
                lines.append(f"| {r['name']} | {r['tol_f1']:.3f} | {r['tol_precision']:.2f}
| "
                             f"{r['tol_recall']:.2f} | {r['dstart_abs_median']:.2f} | "
                             f"{r['dend_abs_median']:.2f} | {int(r['split_count'])} |
{int(r['merge_count'])} |")
            lines += ["", f"**R2 aggregate (n={a['n']}):** tolF1 {_ci(a['tol_f1'])} · "
                      f"tolP {_ci(a['tol_precision'])} · tolR {_ci(a['tol_recall'])} · "
                      f"|?start| {_ci(a['dstart_abs_median'])} · |?end|
{_ci(a['dend_abs_median'])} "
                      f"(start?solved / end=the gap)"]
        return "\n".join(lines)


    def _resolve_preset(name: str) -> WindowingPreset:
        if name in PRESETS:
            return PRESETS[name]
        raise SystemExit(f"unknown preset '{name}'; choices: {sorted(PRESETS)}")


```

```
267     def main() -> None:
268         ap = argparse.ArgumentParser(description="Rally-segmentation eval vs golden labels
(offline).")
269         ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
270         ap.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR)
271         ap.add_argument("--preset", default=SERVED.name, help="windowing preset (default:
served)")
272         ap.add_argument("--vs", default=None, help="second preset to diff against --preset")
273         ap.add_argument("--iou", type=float, default=DEFAULT_IOU)
274         ap.add_argument("--min-crossings", type=int, default=0,
275                         help="net-crossings rally-state gate on --preset (0=off; W2
Gate-A)")
276         ap.add_argument("--vs-min-crossings", type=int, default=0,
277                         help="net-crossings gate on --vs (default: same as --min-
crossings)")
278         ap.add_argument("--max-window-duration", type=float, default=None,
279                         help="override bounded-windowing cap (s) on BOTH presets (W1; e.g.
12)")
280         ap.add_argument("--vs-max-window-duration", type=float, default=None,
281                         help="cap (s) on --vs ONLY (default: same as --max-window-duration);
lets a "
282                              "BASELINE (no cap) vs CANDIDATE (capped) promotion check run in
one shot")
283         ap.add_argument("--hard-cap", action="store_true",
284                         help="enforce the cap as a HARD cap on --preset (chop continuously-
active "
285                              "windows at the cap when no inactivity gap exists; W1 hard-cap
fallback)")
286         ap.add_argument("--vs-hard-cap", action="store_true",
287                         help="hard-cap fallback on --vs (default: same as --hard-cap)")
288         ap.add_argument("--blob-fallback", action="store_true",
289                         help="R5 blob-fallback on --preset (after R4, force-chop any group
still over the "
290                              "cap at its midpoint + flag/log it; the continuously-active
blob last resort)")
291         ap.add_argument("--vs-blob-fallback", action="store_true",
292                         help="R5 blob-fallback on --vs (default: same as --blob-fallback)")
293         ap.add_argument("--blob-factor", type=float, default=None,
294                         help="R5 magnitude gate (both presets): chop only groups > this ×
cap (default 4.0)")
295         ap.add_argument("--inactivity-end", type=float, default=None,
296                         help="R4 rally-end inactivity trim (s) on --preset (0=off; trims
post-rally drift tail)")
297         ap.add_argument("--vs-inactivity-end", type=float, default=None,
298                         help="R4 inactivity trim on --vs (default: same as --inactivity-
end)")
299         ap.add_argument("--inactivity-rally-thresh", type=float, default=None,
300                         help="R4 velocity above which motion counts as 'rally' (both
presets; default 0.08)")
301         ap.add_argument("--inactivity-min-density", type=float, default=None,
302                         help="R4 min trailing fast-fraction to stay 'in rally' (both
presets; default 0.34)")
303         ap.add_argument("--tau-start", type=float, default=None,
304                         help="R2 tolerance-match start window (s); default 2.0 (forgives the
service window)")
305         ap.add_argument("--tau-end", type=float, default=None,
306                         help="R2 tolerance-match end window (s); default 1.5 (keeps the
rally-end honest)")
307         ap.add_argument("--guard-weight-merge", type=float, default=None,
308                         help="R1 over-seg guard: merge cost weight (default 2.0; merge hides
a rally)")
309         ap.add_argument("--guard-weight-split", type=float, default=None,
310                         help="R1 over-seg guard: split cost weight (default 1.0; split only
fragments)")
311         ap.add_argument("--guard-counts", action="store_true",
```

```
                        help="R1 over-seg guard scores raw counts instead of normalized
rates "
                             "(rates are the default ? comparable across a structural
mega?bounded change)")
        ap.add_argument("--json-out", default=None, help="optional path to write the full
result JSON")
        args = ap.parse_args()

        from dataclasses import replace
        golden = load_golden(args.manifest, args.features_dir)
        def _apply_inactivity(preset, end_val):
            if end_val is not None:
                preset = replace(preset, inactivity_end=end_val)
            if args.inactivity_rally_thresh is not None:
                preset = replace(preset,
inactivity_rally_thresh=args.inactivity_rally_thresh)
            if args.inactivity_min_density is not None:
                preset = replace(preset, inactivity_min_density=args.inactivity_min_density)
            return preset

        base = _resolve_preset(args.preset)
        if args.max_window_duration is not None:
            base = replace(base, max_window_duration=args.max_window_duration)
        if args.hard_cap:
            base = replace(base, hard_cap=True)
        if args.blob_fallback:
            base = replace(base, blob_fallback=True)
        if args.blob_factor is not None:
            base = replace(base, blob_factor=args.blob_factor)
        base = _apply_inactivity(base, args.inactivity_end)
        if args.vs:
            vs_mc = args.vs_min_crossings or args.min_crossings
            cand = _resolve_preset(args.vs)
            cand_cap = (args.vs_max_window_duration if args.vs_max_window_duration is not
None
                        else args.max_window_duration)
            if cand_cap is not None:
                cand = replace(cand, max_window_duration=cand_cap)
            if args.vs_hard_cap or args.hard_cap:
                cand = replace(cand, hard_cap=True)
            if args.vs_blob_fallback or args.blob_fallback:
                cand = replace(cand, blob_fallback=True)
            if args.blob_factor is not None:
                cand = replace(cand, blob_factor=args.blob_factor)
            cand = _apply_inactivity(cand, args.vs_inactivity_end if args.vs_inactivity_end
is not None
                                     else args.inactivity_end)
            result = compare(golden, base, cand, args.iou,
                             base_min_crossings=args.min_crossings,
cand_min_crossings=vs_mc,
                             tau_start=args.tau_start, tau_end=args.tau_end,
                             guard_weight_merge=args.guard_weight_merge,
                             guard_weight_split=args.guard_weight_split,
                             guard_use_rates=not args.guard_counts)
            print(format_report(result["base"]))
            print("\n" + format_report(result["cand"]))
            d = result["delta"]
            print(f"\n## ? ({args.vs} mc={vs_mc} ? {args.preset} mc={args.min_crossings}), "
                  f"n_paired={result['n_paired']}")
            for k, dd in d.items():
                st = dd["sign_test"]
                print(f"  ?{k}: {dd['mean_delta']:+.3f}
[{dd['ci_low']:+.3f},{dd['ci_high']:+.3f}] "
                      f"CI?0={dd['ci_excludes_zero']} sign(W/L)={st['wins']}/{st['losses']}
p={st['p_value']:.3f}")
```

```
            print(_format_guard(result["over_seg_guard"]))
        else:
            result = evaluate(golden, base, args.iou, args.min_crossings,
                              tau_start=args.tau_start, tau_end=args.tau_end)
            print(format_report(result))
        if args.json_out:
            with open(args.json_out, "w", encoding="utf-8") as fh:
                json.dump(result, fh, indent=2)
            print(f"\n[written {args.json_out}]")


    if __name__ == "__main__":
        main()

```

5. assistant (2026-06-18T07:47:55.036Z)

Let me read the remaining target files in parallel.

6. user (2026-06-18T07:47:55.999Z)

```
      """SERVED Gate-A verification: run the *production* ``FusionSegmenter`` decision seams
      on a cached WASB trajectory and score the resulting AI-handoff windows vs golden labels.

      The offline litmus (``backend.eval.ablation`` / ``fusion_compare``) measures Gate A on
the
      **permissive proposal windowing** (``distill_local.PROPOSAL`` = recall-oriented) re-
scored
      through the experiment harness's ``_gate_mask``. That validates the *signal*. It does
NOT
      exercise the **served path**: in production the windows come from
      ``trajectory_to_action_windows`` at the production operating point
      (``velocity_threshold=0.02, dead_time_merge_gap=2.0, min_action_window_duration=2.0``),
the
      net-crossing count is computed by ``FusionSegmenter._enrich_candidate_stats``, and the
gate is
      ``FusionSegmenter._select_for_handoff``. THIS module drives exactly those production
methods ?
      no reimplementation of the windowing or the gate ? on a cached trajectory CSV, so the
      verification reflects what ``process_video`` actually hands to Gemini.

      GPU-free and Gemini-free: it stops at the handoff-selection boundary (the windows that
WOULD
      be sent to the AI). With the person cue OFF (the default) nothing here loads torch or
touches
      the GPU ? it just re-scores cached WASB trajectories.

      Run (owner box, golden trajectories present):
          python -m backend.eval.served_gate_a --manifest output/human_lovo_manifest.json
      """
      from __future__ import annotations

      import argparse
      import json
      from dataclasses import dataclass
      from typing import Any, Dict, List, Optional, Tuple

      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
      from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
      from backend.eval.calibrate_wasb import load_golden_gts
      from backend.eval.metrics import Interval, score, Score
      from backend.eval import segmentation_metrics as _segm

      # Production windowing operating point (config.json indexing defaults ? the values
      # TrajectoryHybridSegmenter.process_video reads for the served fusion run).
```

```
    PROD_VELOCITY_THRESH = 0.02
    PROD_MERGE_GAP = 2.0
    PROD_MIN_WINDOW = 2.0


    def _served_idx_cfg(min_crossings: int) -> Dict[str, Any]:
        """The ``indexing`` config dict the served FusionSegmenter seams read ? mirrors
        config.json's ``indexing.fusion`` block, with Gate A at ``min_crossings`` and the
        person cue OFF (so no GPU / torch is touched)."""
        return {
            "fusion": {
                "substrate": "wasb",
                "min_crossings": min_crossings,
                "min_players": 0,
                "cue_flags": {},
            },
            "wasb": {"velocity_threshold": PROD_VELOCITY_THRESH},
            "dead_time_merge_gap": PROD_MERGE_GAP,
            "min_action_window_duration": PROD_MIN_WINDOW,
        }


    def served_handoff_windows(trajectory_csv: str, fps: float, frame_width: float,
                               min_crossings: int) -> List[Interval]:
        """Run the PRODUCTION FusionSegmenter decision path on a cached trajectory and
return the
        windows that would reach the AI handoff (i.e. the served rally candidates).

        Uses the real ``FusionSegmenter`` seam methods ? ``_enrich_candidate_stats``
(computes
        ``net_crossings``) and ``_select_for_handoff`` (Gate A) ? over windows from the
production
        ``trajectory_to_action_windows`` operating point. No GPU, no Gemini: the person cue
is OFF.
        """
        points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
        windows = TrackNetRunner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width,
            velocity_thresh=PROD_VELOCITY_THRESH, merge_gap=PROD_MERGE_GAP,
            min_window_duration=PROD_MIN_WINDOW,
        )
        idx_cfg = _served_idx_cfg(min_crossings)
        # Construct the real served segmenter (db=None, config carries the indexing block).
We only
        # call the pure decision seams ? no process_video, no DB, no provider ? exactly as
the
        # registry constructs it for `name`/`description`.
        seg = FusionSegmenter(None, {"indexing": idx_cfg, "sport": "badminton"})
        # The served per-window stats (incl. net_crossings) via the production enrichment
hook.
        window_stats = [
            seg._enrich_candidate_stats(cw, points, fps, frame_width, video_path="",
                                        idx_cfg=idx_cfg)
            for cw in windows
        ]
        keep = seg._select_for_handoff(windows, window_stats, idx_cfg)
        return [(windows[i]["start"], windows[i]["end"]) for i in keep]


    @dataclass
    class ServedVideoResult:
        name: str
        num_gt: int
        on: Score                # Gate-A ON  (min_crossings=2)
        off: Score               # Gate-A OFF (min_crossings=0, control)
```

```
        seg_ratio_on: float       # |preds| / |gts| with Gate-A ON  (over-segmentation)
        seg_ratio_off: float
        n_handoff_on: int
        n_handoff_off: int

        @property
        def delta_f1(self) -> float:
            return self.on.f1 - self.off.f1


    def evaluate_video(spec: Dict[str, Any], iou: float = 0.5,
                       on_crossings: int = 2) -> ServedVideoResult:
        """Score the served Gate-A ON (min_crossings=on_crossings) vs OFF (0) handoff
windows for
        one golden video against its golden labels."""
        fps, fw = float(spec["fps"]), float(spec["frame_width"])
        traj, labels = str(spec["trajectory"]), str(spec["labels"])
        gts = load_golden_gts(labels)
        preds_on = served_handoff_windows(traj, fps, fw, min_crossings=on_crossings)
        preds_off = served_handoff_windows(traj, fps, fw, min_crossings=0)
        return ServedVideoResult(
            name=str(spec["name"]),
            num_gt=len(gts),
            on=score(preds_on, gts, iou),
            off=score(preds_off, gts, iou),
            seg_ratio_on=_segm.segment_count_ratio(preds_on, gts),
            seg_ratio_off=_segm.segment_count_ratio(preds_off, gts),
            n_handoff_on=len(preds_on),
            n_handoff_off=len(preds_off),
        )


    def evaluate_manifest(specs: List[Dict[str, Any]], iou: float = 0.5,
                          on_crossings: int = 2) -> List[ServedVideoResult]:
        return [evaluate_video(s, iou, on_crossings) for s in specs]


    def aggregate(results: List[ServedVideoResult]) -> Dict[str, Any]:
        """Mean ON/OFF F1 + over-seg ratio, the paired ?F1, and the per-video win count."""
        n = len(results)

        def _mean(getter) -> float:
            return sum(getter(r) for r in results) / n if n else 0.0

        deltas = [r.delta_f1 for r in results]
        wins = sum(1 for d in deltas if d > 1e-9)
        losses = sum(1 for d in deltas if d < -1e-9)
        return {
            "num_videos": n,
            "mean_f1_on": _mean(lambda r: r.on.f1),
            "mean_f1_off": _mean(lambda r: r.off.f1),
            "mean_delta_f1": _mean(lambda r: r.delta_f1),
            "mean_precision_on": _mean(lambda r: r.on.precision),
            "mean_precision_off": _mean(lambda r: r.off.precision),
            "mean_recall_on": _mean(lambda r: r.on.recall),
            "mean_recall_off": _mean(lambda r: r.off.recall),
            "mean_seg_ratio_on": _mean(lambda r: r.seg_ratio_on),
            "mean_seg_ratio_off": _mean(lambda r: r.seg_ratio_off),
            "wins": wins,
            "losses": losses,
        }


    def format_table(results: List[ServedVideoResult], agg: Dict[str, Any], iou: float) ->
str:
```

```
158         lines = [
159             f"=== SERVED Gate-A verification (production FusionSegmenter path, IoU>={iou})
===",
160             f"  {'video':<24s} {'gt':>3s} {'F1_off':>7s} {'F1_on':>7s} {'?F1':>7s} "
161             f"{'segR_off':>9s} {'segR_on':>8s} {'hand_off':>9s} {'hand_on':>8s}",
162         ]
163         for r in results:
164             lines.append(
165                 f"  {r.name:<24s} {r.num_gt:>3d} {r.off.f1:>7.3f} {r.on.f1:>7.3f} "
166                 f"{r.delta_f1:>+7.3f} {r.seg_ratio_off:>9.2f} {r.seg_ratio_on:>8.2f} "
167                 f"{r.n_handoff_off:>9d} {r.n_handoff_on:>8d}")
168         lines += [
169             f"  {'-' * 92}",
170             f"  MEAN  F1 {agg['mean_f1_off']:.3f} (off) -> {agg['mean_f1_on']:.3f} (on)  "
171             f"?F1={agg['mean_delta_f1']:+.3f}   "
172             f"P {agg['mean_precision_off']:.3f}->{agg['mean_precision_on']:.3f}  "
173             f"R {agg['mean_recall_off']:.3f}->{agg['mean_recall_on']:.3f}",
174             f"  over-seg ratio {agg['mean_seg_ratio_off']:.2f} (off) -> "
175             f"{agg['mean_seg_ratio_on']:.2f} (on)  | wins {agg['wins']}/{agg['num_videos']}
"
176             f"(losses {agg['losses']})",
177         ]
178         return "\n".join(lines)
179
180
181     def run(manifest_path: str, iou: float = 0.5) -> Tuple[List[ServedVideoResult],
Dict[str, Any]]:
182         with open(manifest_path, encoding="utf-8") as f:
183             specs = json.load(f)
184         results = evaluate_manifest(specs, iou=iou)
185         return results, aggregate(results)
186
187
188     def main(argv: Optional[List[str]] = None) -> None:
189         ap = argparse.ArgumentParser(
190             description="Verify served Gate-A on real footage via the production
FusionSegmenter path.")
191         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
192         ap.add_argument("--iou", type=float, default=0.5)
193         args = ap.parse_args(argv)
194         results, agg = run(args.manifest, iou=args.iou)
195         print(format_table(results, agg, args.iou))
196
197
198     if __name__ == "__main__":
199         main()
200
```

7. user (2026-06-18T07:47:56.650Z)

```
1       """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3       **The bug this closes.** The offline eval substrate (``distill_local.build_candidates``
?
4       ``fusion_golden`` ? the golden feature JSONs the ablation/compare path re-scores) was
built
5       on a *recall-oriented PROPOSAL* windowing (``vt=0.005, merge_gap=1.0, min_window=0.5``):
6       deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
pipeline
7       (``TrajectoryHybridSegmenter.process_video``) windows COARSELY (``vt=0.02,
merge_gap=2.0,
8       min_window=2.0`` ? the ``config.indexing`` defaults). A cue measured on the proposal set
is
9       measured on candidates **production never serves**, so a "win" need not transfer. This
is
```

```
    exactly how Gate-A scored **+0.074** on the proposal set yet **?0.008** on the served
set
    (``scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md`` vs ``output/gateA_served_verify/``).

    **The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and
the
    SERVED preset is **single-sourced from the same Pydantic config defaults production
reads**
    (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
Helpers
    build candidates under either preset, and :func:`served_windowing_from_config` lifts the
    *actual* served windowing out of a live config (overrides included), so an A/B can
target the
    operating point a given deployment truly serves.

    The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
    windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval_stats`` +
    ``per_user_gate``). This module is the **what windowing**; that one is the **promotion
gate**.

    Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
    caller opts in (``distill_local`` re-points its module-level presets here, byte-
identically).
    """
    from __future__ import annotations

    from dataclasses import dataclass
    from typing import Any, Dict, List, Mapping, Tuple

    from backend.config.models import IndexingConfig

    Interval = Tuple[float, float]

    __all__ = [
        "WindowingPreset",
        "SERVED",
        "PROPOSAL",
        "PRESETS",
        "served_windowing_from_config",
        "build_windows",
        "windows_to_intervals",
    ]


    @dataclass(frozen=True)
    class WindowingPreset:
        """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.

        ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
        "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
        ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
        knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
windowing.
        """

        name: str
        velocity_thresh: float
        merge_gap: float
        min_window_duration: float
        #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
```

```
internal
62          #: inactivity gap ? the bounded-windowing over-merge fix (W1,
RALLY_QUALITY_RESEARCH.md §6).
63          max_window_duration: float = 0.0
64          #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
internal
65          #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active
fix). OFF by default.
66          hard_cap: bool = False
67          #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift
tail back to
68          #: the last frame whose trailing window stays dense with FAST motion. The measured
rally-end fix.
69          inactivity_end: float = 0.0
70          #: velocity above which motion counts as "rally" (vs slow drift) for the R4 trim;
min fast-fraction.
71          inactivity_rally_thresh: float = 0.08
72          inactivity_min_density: float = 0.34
73          #: R5 blob-fallback (default OFF). After the cap split + R4 trim, midpoint-chop any
group still
74          #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived
blob ?
75          #: mahadevpura-1's ~11× residual) and flag it. Runs AFTER R4 (vs ``hard_cap``
before) and the
76          #: factor keeps it surgical (a normal long rally is left alone).
77          blob_fallback: bool = False
78          blob_factor: float = 4.0
79
80          def as_tuple(self) -> Tuple[float, float, float]:
81              """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
82              ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
83              bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
84              return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
85
86          def to_dict(self) -> Dict[str, Any]:
87              return {
88                  "name": self.name,
89                  "velocity_thresh": self.velocity_thresh,
90                  "merge_gap": self.merge_gap,
91                  "min_window_duration": self.min_window_duration,
92                  "max_window_duration": self.max_window_duration,
93                  "hard_cap": self.hard_cap,
94                  "inactivity_end": self.inactivity_end,
95                  "inactivity_rally_thresh": self.inactivity_rally_thresh,
96                  "inactivity_min_density": self.inactivity_min_density,
97                  "blob_fallback": self.blob_fallback,
98                  "blob_factor": self.blob_factor,
99              }
100
101
102     def _served_preset_from_defaults() -> WindowingPreset:
103         """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
104
105         ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
106           - velocity:   ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
107           - merge_gap:  ``idx_cfg.dead_time_merge_gap`` (default 2.0)
108           - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
109
110         Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
```

```
        point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
        so the two can never silently diverge again.
        """
        idx = IndexingConfig()  # construct with defaults ? the served operating point
        return WindowingPreset(
            name="served",
            velocity_thresh=float(idx.wasb.velocity_threshold),  # == fusion/tracknet
default 0.02
            merge_gap=float(idx.dead_time_merge_gap),
            min_window_duration=float(idx.min_action_window_duration),
            max_window_duration=float(idx.max_window_duration),  # W1 bounded-windowing
(default 0=OFF)
            hard_cap=bool(idx.window_hard_cap),                   # W1 hard-cap fallback
(default OFF)
            inactivity_end=float(idx.window_inactivity_end),      # R4 rally-end trim
(default 0=OFF)
            inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
            inactivity_min_density=float(idx.inactivity_min_density),
            blob_fallback=bool(idx.window_blob_fallback),         # R5 blob-fallback
(default OFF)
            blob_factor=float(idx.window_blob_factor),
        )


    #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
    #: only promotable if it holds up HERE ? the operating point real users get.
    SERVED: WindowingPreset = _served_preset_from_defaults()

    #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
    #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
    #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
    #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
    PROPOSAL: WindowingPreset = WindowingPreset(
        name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
    )

    #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
    PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}


    def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
        """The SERVED windowing a *specific* live config serves (overrides honoured).

        Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
        arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
        (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
        default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
        this is *this config's* operating point. Falls back to the served defaults for any
absent key.
        """
        idx = _as_mapping(config, "indexing")
        family_cfg = _as_mapping(idx, family)
        return WindowingPreset(
            name=f"served:{family}",
            velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
```

```
160             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
161             min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
162             max_window_duration=float(idx.get("max_window_duration",
SERVED.max_window_duration)),
163             hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
164             inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
165             inactivity_rally_thresh=float(idx.get("inactivity_rally_thresh",
SERVED.inactivity_rally_thresh)),
166             inactivity_min_density=float(idx.get("inactivity_min_density",
SERVED.inactivity_min_density)),
167             blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
168             blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
169         )
170
171
172     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
173         """``obj[key]`` as a plain mapping, whether ``obj`` is a dict or a typed config
section
174         (``AppConfig.get`` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
175         if obj is None:
176             return {}
177         if isinstance(obj, Mapping):
178             sub = obj.get(key, {})
179         elif hasattr(obj, "get"):
180             sub = obj.get(key, {})
181         else:
182             sub = getattr(obj, key, {})
183         return sub if isinstance(sub, Mapping) else {}
184
185
186     def build_windows(points: List[Any], fps: float, frame_width: float,
187                       preset: WindowingPreset) -> List[Dict[str, Any]]:
188         """Trajectory ``points`` ? candidate windows under ``preset`` (the one windowing
call site).
189
190         A thin wrapper over ``TrackNetRunner.trajectory_to_action_windows`` that takes a
named preset
191         instead of three loose floats, so eval candidate-building and the served path
reference the
192         SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
193         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
194
195         vt, mg, mwd = preset.as_tuple()
196         return TrackNetRunner.trajectory_to_action_windows(
197             points, fps=fps, frame_width=frame_width,
198             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
199             max_window_duration=preset.max_window_duration,
200             window_hard_cap=preset.hard_cap,
201             window_inactivity_end=preset.inactivity_end,
202             inactivity_rally_thresh=preset.inactivity_rally_thresh,
203             inactivity_min_density=preset.inactivity_min_density,
204             window_blob_fallback=preset.blob_fallback,
205             window_blob_factor=preset.blob_factor)
206
207
208     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
209         """``[{start, end, ...}]`` ? ``[(start, end)]`` ? the interval shape the scorer/gate
speak."""
210         return [(w["start"], w["end"]) for w in windows]
211
```

8. assistant (2026-06-18T07:47:59.230Z)

Now let me read the remaining files: experiment.py, per_user_gate.py, promotion.py, regression.py.

9. user (2026-06-18T07:48:00.315Z)

```
1       """Per-user promotion gate ? the safety rail for flag-gated personalization.
2
3       The owner adopted personalization as a **flag-gated** feature on a **generalization-
first**
4       baseline: a cue/variant B may be flipped ON *for one user* only when that user's own
5       held-out evidence honestly clears the same small-N bar the global eval uses ? otherwise
we
6       fall back to the shipped baseline (variant A). This module is the decision logic for
that
7       flip; it is pure (no I/O) and **reuses** ``backend.eval.eval_stats`` for every statistic
8       (``paired_delta_ci`` + ``paired_sign_test``) so the math is never reinvented.
9
10      The counter-case this guards against (and its mitigations, implemented here):
11
12      1. **Minimum-N floor.** Personalizing off 1-2 of a user's videos is the small-N-lies
regime
13         (see ``eval_stats`` C1). Refuse to promote below ``min_n`` videos ? point estimates
from
14         too few held-out videos are noise, not signal.
15      2. **Honest lift, not a single mean.** Promote only when the paired ?F1 95% bootstrap CI
16         **excludes 0** *and* the paired sign test agrees (majority of videos win, p <=
threshold).
17         This is the global promotion bar applied per user.
18      3. **Structural guards are never disabled on "no lift".** A *structural* guard (e.g.
Gate-A
19         net-crossings ? the held-shuttle false-positive killer) earns its keep through
failure
20         modes a given user's corpus may not contain yet. Measuring "no per-user lift" on a
corpus
21         that simply lacks those failure cases must NEVER turn the guard OFF. For
``structural=True``
22         the decision is therefore **always "keep ON"**, independent of the measured delta.
23
24      Below the floor / insufficient evidence ? fall back to the shipped baseline (the
conservative
25      default), never an unsafe promotion. All additive, pure, deterministic (the bootstrap
seed is
26      threaded through). Nothing here changes existing eval behaviour; callers opt in.
27      """
28      from __future__ import annotations
29
30      from dataclasses import dataclass, field
31      from typing import Any, Dict, Sequence
32
33      from backend.eval.eval_stats import paired_delta_ci, paired_sign_test
34
35      __all__ = ["PromotionDecision", "should_promote_for_user"]
36
37
38      @dataclass
39      class PromotionDecision:
40          """Structured result of a per-user promotion decision.
41
42          - ``promote`` ? flip variant B ON for this user (only ever ``True`` for a non-
structural
43            cue that cleared the full bar).
44          - ``keep_on`` ? the cue/guard should be ON for this user. For a structural guard
this is
45            always ``True`` (it is never disabled on "no lift"); for a non-structural cue it
equals
```

```
46            ``promote``.
47          - ``reason`` ? short human-readable explanation of the decision.
48          - the remaining fields surface the evidence (n, mean delta, CI bounds and the two
tests)
49            so a caller can log/audit exactly why a user was or wasn't personalized.
50          """
51
52          promote: bool
53          keep_on: bool
54          reason: str
55          n: int
56          mean_delta: float
57          ci_low: float
58          ci_high: float
59          ci_excludes_zero: bool
60          sign_test: Dict[str, float] = field(default_factory=dict)
61          floor_met: bool = False
62          structural_protected: bool = False
63
64          def as_dict(self) -> Dict[str, Any]:
65              """Plain-dict view (for JSON logging / telemetry)."""
66              return {
67                  "promote": self.promote,
68                  "keep_on": self.keep_on,
69                  "reason": self.reason,
70                  "n": self.n,
71                  "mean_delta": self.mean_delta,
72                  "ci_low": self.ci_low,
73                  "ci_high": self.ci_high,
74                  "ci_excludes_zero": self.ci_excludes_zero,
75                  "sign_test": self.sign_test,
76                  "floor_met": self.floor_met,
77                  "structural_protected": self.structural_protected,
78              }
79
80
81      def should_promote_for_user(
82          a_scores: Sequence[float],
83          b_scores: Sequence[float],
84          *,
85          structural: bool = False,
86          min_n: int = 5,
87          p_threshold: float = 0.05,
88          confidence: float = 0.95,
89          n_boot: int = 2000,
90          seed: int = 0,
91          eps: float = 0.0,
92      ) -> PromotionDecision:
93          """Decide whether to flip variant B ON for ONE user from that user's held-out
scores.
94
95          ``a_scores`` / ``b_scores`` are the user's **per-video** held-out F1 (or any score)
for
96          variant A (control / shipped baseline) and variant B (with the cue), aligned by
video
97          (same order). The decision rules (faithful to the module docstring):
98
99          1. ``structural=True`` ? the guard is **never disabled on "no lift"**: return
100             ``keep_on=True, promote=False`` regardless of the measured per-user delta. Its
value is
101             in failure modes the user's corpus may not yet contain.
102         2. Otherwise promote **iff ALL** hold ? the minimum-N floor (``n >= min_n``), the
paired
103             ?F1 bootstrap CI **excludes 0**, and the paired sign test agrees (majority of
videos
```

```
104            win, ``p_value <= p_threshold``). Any miss ? fall back to the shipped baseline.
105
106        Statistics come straight from ``eval_stats`` (``paired_delta_ci`` /
``paired_sign_test``);
107        deterministic given ``seed``. Returns a :class:`PromotionDecision`.
108        """
109        n = min(len(a_scores), len(b_scores))
110        floor_met = n >= min_n
111
112        # --- Evidence (reuse eval_stats; never reinvent the statistics)
--------------------
113        if n == 0:
114            # No paired videos at all ? nothing to test. Surface an empty-but-honest result.
115            sign_test: Dict[str, float] = paired_sign_test([], eps)
116            mean_delta, ci_low, ci_high, ci_excludes_zero = 0.0, 0.0, 0.0, False
117        else:
118            delta = paired_delta_ci(
119                a_scores, b_scores,
120                confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,
121            )
122            mean_delta = float(delta["mean_delta"])
123            ci_low = float(delta["ci_low"])
124            ci_high = float(delta["ci_high"])
125            ci_excludes_zero = bool(delta["ci_excludes_zero"])
126            sign_test = delta["sign_test"]
127
128        sign_p = float(sign_test.get("p_value", 1.0))
129        sign_wins = float(sign_test.get("wins", 0.0))
130        sign_losses = float(sign_test.get("losses", 0.0))
131        # The sign test "agrees" with a promotion only if it is significant AND the majority
132        # leans the *winning* way (more B-wins than B-losses) ? a significant net *loss*
must
133        # never satisfy the promotion bar.
134        sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
135
136        # --- Rule 1: structural guards are never disabled on "no lift"
----------------------
137        if structural:
138            return PromotionDecision(
139                promote=False,           # nothing to "promote" ? it ships ON by design
140                keep_on=True,            # ALWAYS kept ON, regardless of measured per-user
lift
141                reason=(
142                    "structural guard kept ON regardless of measured per-user lift "
143                    "(value is in failure modes this user's corpus may not yet contain)"
144                ),
145                n=n,
146                mean_delta=mean_delta,
147                ci_low=ci_low,
148                ci_high=ci_high,
149                ci_excludes_zero=ci_excludes_zero,
150                sign_test=sign_test,
151                floor_met=floor_met,
152                structural_protected=True,
153            )
154
155        # --- Rule 2: minimum-N floor (refuse before even looking at lift)
--------------------
156        if not floor_met:
157            return PromotionDecision(
158                promote=False,
159                keep_on=False,           # fall back to the shipped baseline (variant A)
160                reason=(
161                    f"below minimum-N floor (n={n} < min_n={min_n}); "
162                    "falling back to shipped baseline"
```

```
163                 ),
164                 n=n,
165                 mean_delta=mean_delta,
166                 ci_low=ci_low,
167                 ci_high=ci_high,
168                 ci_excludes_zero=ci_excludes_zero,
169                 sign_test=sign_test,
170                 floor_met=False,
171                 structural_protected=False,
172             )
173 
174         # --- Rule 2 (cont.): honest lift ? CI excludes 0 AND sign test agrees
----------------
175         if ci_excludes_zero and sign_agrees:
176             promote = True
177             reason = (
178                 f"promoted: n={n} >= {min_n}, ?F1 95% CI "
179                 f"[{ci_low:.4f}, {ci_high:.4f}] excludes 0, "
180                 f"sign test {int(sign_wins)}-{int(sign_losses)} (p={sign_p:.4f} <=
{p_threshold})"
181             )
182         else:
183             promote = False
184             misses = []
185             if not ci_excludes_zero:
186                 misses.append(f"?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] spans 0")
187             if not sign_agrees:
188                 misses.append(
189                     f"sign test does not agree ({int(sign_wins)}-{int(sign_losses)}, "
190                     f"p={sign_p:.4f})"
191                 )
192             reason = (
193                 "insufficient evidence (" + "; ".join(misses) + "); "
194                 "falling back to shipped baseline"
195             )
196 
197         return PromotionDecision(
198             promote=promote,
199             keep_on=promote,             # non-structural: keep_on tracks promote exactly
200             reason=reason,
201             n=n,
202             mean_delta=mean_delta,
203             ci_low=ci_low,
204             ci_high=ci_high,
205             ci_excludes_zero=ci_excludes_zero,
206             sign_test=sign_test,
207             floor_met=True,
208             structural_protected=False,
209         )
210 
```

10. user (2026-06-18T07:48:00.991Z)

```
1       """Promotion gate with the eval?serve guard: a cue is promotable ONLY if it does not
regress
2       at the SERVED windowing.
3 
4       **Why this exists.** ``per_user_gate.should_promote_for_user`` already grades a B?A lift
5       honestly (min-N floor + ?F1 CI excludes 0 + paired sign test). But it grades whatever
scores
6       it is handed ? and the foundational bug was that those scores came from the *recall-
oriented
7       PROPOSAL* windowing (``backend.eval.windowing.PROPOSAL``), which production never
serves. So a
8       cue could clear the bar on a windowing nobody runs (Gate-A: proposal ?F1 **+0.074**)
```

```
while
9       regressing on the served one (**?0.008**). Grading harder isn't the fix; **measuring on
the
10      served windowing** is.
11
12      This module composes ? it adds no new statistics. It layers ONE rule on top of
13      ``should_promote_for_user``:
14
15          A cue is promotable only if, at the **SERVED** windowing, it WINS or at least DOES
NOT
16          REGRESS (its served paired ?F1 mean ? ``-served_regress_eps``, and ? when its served
CI is
17          estimable ? that CI's lower bound does not sit clearly in regression territory).
18
19      A proposal-windowing win is still *reported* (it's useful signal for where to look
next), but it
20      can never carry a promotion on its own. The decision returns BOTH the served and the
proposal
21      verdicts so the eval?serve contrast is explicit and auditable.
22
23      Pure, deterministic, stdlib-only over ``eval_stats``/``per_user_gate``. Additive:
existing eval
24      paths are untouched until a caller routes its A/B through
:func:`promote_if_served_safe`.
25      """
26      from __future__ import annotations
27
28      from dataclasses import dataclass, field
29      from typing import Any, Dict, Optional, Sequence
30
31      from backend.eval.eval_stats import paired_delta_ci
32      from backend.eval.per_user_gate import PromotionDecision, should_promote_for_user
33
34      __all__ = ["ServedGateResult", "promote_if_served_safe", "served_regresses"]
35
36
37      def served_regresses(a_scores: Sequence[float], b_scores: Sequence[float], *,
38                           eps: float = 0.0, n_boot: int = 2000, seed: int = 0) -> Dict[str,
Any]:
39          """Does variant B REGRESS variant A at the served windowing? (the eval?serve guard
core).
40
41          ``a_scores``/``b_scores`` are per-video held-out scores **measured at the SERVED
windowing**
42          (aligned by video). Returns the paired B?A summary (mean ?F1 + bootstrap CI + sign
test) plus
43          a single ``regresses`` boolean:
44
45            - ``regresses=True``  when the served mean ?F1 is below ``-eps`` (a real served
drop), OR
46              the served ?F1 CI lies **entirely** below ``-eps`` (a confidently-negative
band).
47            - ``regresses=False`` when the cue holds the line at the served operating point
(mean ?
48              ``-eps`` and the CI is not a confident net-negative).
49
50          ``eps`` is the regression tolerance: 0.0 = "must not drop at all on the served
mean". A tiny
51          positive ``eps`` lets noise-level wobble through (e.g. 0.005 ? half an F1 point at
n?6).
52          """
53          delta = paired_delta_ci(a_scores, b_scores, n_boot=n_boot, seed=seed)
54          mean_delta = float(delta["mean_delta"])
55          ci_low = float(delta["ci_low"])
56          ci_high = float(delta["ci_high"])
```

```
        # Regression if the served mean dropped beyond tolerance, OR the whole CI is net-negative.
        mean_regresses = mean_delta < -eps
        ci_confidently_negative = ci_high < -eps
        regresses = bool(mean_regresses or ci_confidently_negative)
        return {
            "regresses": regresses,
            "mean_delta": mean_delta,
            "ci_low": ci_low,
            "ci_high": ci_high,
            "ci_excludes_zero": bool(delta["ci_excludes_zero"]),
            "sign_test": delta["sign_test"],
            "n": int(delta["n"]),
            "eps": float(eps),
        }


    @dataclass
    class ServedGateResult:
        """A promotion decision that BOTH measures the cue on the served windowing AND reports the
        proposal-windowing view, so the eval?serve contrast is never hidden.

        - ``promote`` ? final verdict. ``True`` only when the underlying honest gate would promote
          *and* the served check did not veto (the cue does not regress at the served windowing).
        - ``served_safe`` ? the eval?serve guard's verdict in isolation (cue does not regress served).
        - ``vetoed_by_served`` ? ``True`` when the proposal/base gate WOULD have promoted but the
          served regression check overruled it (the exact Gate-A failure mode this module prevents).
        - ``base_decision`` ? the ``PromotionDecision`` from ``should_promote_for_user`` on the
          *promotion-grade* scores (served by default ? see :func:`promote_if_served_safe`).
        - ``served`` / ``proposal`` ? the paired B?A summary at each windowing (means + CIs + sign
          tests), so a reviewer sees "+0.074 proposal / ?0.008 served" side by side.
        - ``reason`` ? short human-readable explanation of the final verdict.
        """

        promote: bool
        served_safe: bool
        vetoed_by_served: bool
        reason: str
        base_decision: PromotionDecision
        served: Dict[str, Any]
        proposal: Optional[Dict[str, Any]] = field(default=None)

        def as_dict(self) -> Dict[str, Any]:
            return {
                "promote": self.promote,
                "served_safe": self.served_safe,
                "vetoed_by_served": self.vetoed_by_served,
                "reason": self.reason,
                "base_decision": self.base_decision.as_dict(),
                "served": self.served,
                "proposal": self.proposal,
            }


    def promote_if_served_safe(
        served_a: Sequence[float],
        served_b: Sequence[float],
```

```
        *,
        proposal_a: Optional[Sequence[float]] = None,
        proposal_b: Optional[Sequence[float]] = None,
        structural: bool = False,
        min_n: int = 5,
        p_threshold: float = 0.05,
        served_regress_eps: float = 0.0,
        n_boot: int = 2000,
        seed: int = 0,
    ) -> ServedGateResult:
        """Promote a cue ONLY if it does not regress at the SERVED windowing (the
foundational rule).

        The honest bar (``should_promote_for_user``: min-N floor + ?F1 CI excludes 0 + sign
test) is
        applied to the **served** scores ? the operating point production runs ? so a
proposal-only
        win can never carry a promotion. The optional ``proposal_*`` scores are scored too
and
        reported alongside, purely to surface the eval?serve contrast (they NEVER promote on
their
        own).

        Args:
          served_a/served_b: per-video held-out scores for variant A (control) and B (the
cue),
            measured at the **SERVED** windowing. These drive the decision.
          proposal_a/proposal_b: the same scores at the PROPOSAL windowing (optional).
Reported for
            contrast; the served regression guard still has the final say.
          structural: forwarded to ``should_promote_for_user`` ? a structural guard (e.g.
Gate-A vs
            held-shuttle FPs) is never *disabled* on "no lift", but it is ALSO not
auto-*promoted* to
            the served default if it regresses served. So for ``structural=True`` we report
            ``keep_on`` honestly but ``promote`` reflects the served-safety check.
          served_regress_eps: served regression tolerance (0.0 = "no served-mean drop
allowed").

        Returns a :class:`ServedGateResult` with the final verdict + both windowings'
evidence.
        """
        # 1) The honest gate, applied to the SERVED scores (never the proposal ones).
        base = should_promote_for_user(
            served_a, served_b,
            structural=structural, min_n=min_n, p_threshold=p_threshold,
            n_boot=n_boot, seed=seed,
        )

        # 2) The eval?serve guard: does B regress A at the served windowing?
        served = served_regresses(served_a, served_b, eps=served_regress_eps,
                                  n_boot=n_boot, seed=seed)
        served_safe = not served["regresses"]

        # 3) Optional proposal-windowing view (reported, never promoting on its own).
        proposal: Optional[Dict[str, Any]] = None
        if proposal_a is not None and proposal_b is not None:
            proposal = served_regresses(proposal_a, proposal_b, eps=served_regress_eps,
                                        n_boot=n_boot, seed=seed)

        # 4) Compose the verdict. The served guard can only ever REMOVE a promotion, never
add one.
        if structural:
            # A structural guard ships ON regardless of measured lift (base.keep_on stays
True), but
```

```
165             # it is only auto-promoted to the SERVED default when it is served-safe. If it
regresses
166             # served, promote=False (don't bake an eval-only win into production) while
keep_on holds.
167             promote = served_safe
168             vetoed = False
169             if served_safe:
170                 reason = ("structural guard kept ON; served-safe (does not regress at the
served "
171                           f"windowing, ?F1 {served['mean_delta']:+.4f}) ? eligible for
served default")
172             else:
173                 reason = ("structural guard kept ON, but NOT promoted to the served default:
it "
174                           f"REGRESSES at the served windowing (?F1
{served['mean_delta']:+.4f}); "
175                           "an eval-only win must not be served")
176         else:
177             would_promote = base.promote
178             if would_promote and not served_safe:
179                 promote = False
180                 vetoed = True
181                 reason = ("VETOED by the served-windowing guard: the cue clears the honest
bar but "
182                           f"REGRESSES at the served windowing (served ?F1
{served['mean_delta']:+.4f}"
183                           f", CI [{served['ci_low']:+.4f}, {served['ci_high']:+.4f}]). A win
on a "
184                           "windowing production doesn't serve cannot promote.")
185             elif would_promote:
186                 promote = True
187                 vetoed = False
188                 reason = ("promoted: clears the honest bar AND is served-safe (served ?F1 "
189                           f"{served['mean_delta']:+.4f}). " + base.reason)
190             else:
191                 promote = False
192                 vetoed = False
193                 reason = base.reason
194
195         return ServedGateResult(
196             promote=promote,
197             served_safe=served_safe,
198             vetoed_by_served=vetoed,
199             reason=reason,
200             base_decision=base,
201             served=served,
202             proposal=proposal,
203         )
204
```

11. assistant (2026-06-18T07:48:04.473Z)

Now let me read experiment.py (focusing on the moved `_is_fusion_schema`) and regression.py.

12. user (2026-06-18T07:48:05.893Z)

```
1       """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3       This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4       identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5       no-regression gate, the pinnable classifier artifact, the persisted-candidate
6       substrate) already exists ? this module composes it. **No new scoring math lives
7       here**; everything defers to:
8
9         - `backend.eval.metrics`        ? `score`, `match_intervals`, `interval_iou`
```

```
      - `backend.eval.training`       ? `label_candidates` (y by the same IoU matcher)
      - `backend.eval.classifier`     ? `LogisticRegression` (the pinnable JSON model)
      - `backend.eval.gemini_refine`  ? `merge_overlaps` (the blessed window merge)
      - `backend.eval.regression`     ? `gt_hash` (label-set fingerprint)
      - `backend.eval.calibration`    ? `pct_improvement`

    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
    (per-frame signals ? run once on the GPU/WSL box, persisted into
    `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
    signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
    given persisted candidate features it produces `List[Interval]` deterministically,
    with no GPU and zero production risk. So most experiments are pure offline
    re-scoring of stored data and never touch the served pipeline.

    Three modes (`EXPERIMENT_HARNESS.md` §5):
      - `compare()`           ? labeled A/B vs golden labels: per-video + LOVO-honest
        aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
        held-out loop, but variant-parameterised.
      - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
        variants agree / diverge (A-only, B-only, agreed). The divergences are what a
        human spot-checks (and what two highlight reels would show side by side).
      - the promotion gate stays `run_regression.py` + `regression_baseline.json`
        (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**

    Pure + offline + unit-tested. The only DB-touching helper is
    `load_video_candidates`, which reads persisted candidates via
    `Database.query_candidate_segments`.
    """
    from __future__ import annotations

    import json
    import logging
    import os
    from dataclasses import dataclass, field
    from datetime import datetime, timezone
    from hashlib import sha1
    from typing import Any, Callable, Dict, List, Optional, Sequence

    from backend.eval.calibration import pct_improvement
    from backend.eval.classifier import LogisticRegression
    from backend.eval.gemini_refine import merge_overlaps
    from backend.eval.metrics import Interval, match_intervals, score
    from backend.eval.regression import gt_hash
    from backend.eval.training import label_candidates
    from backend.eval import eval_stats as _stats           # C1: CIs + paired sign test
    from backend.eval import segmentation_metrics as _segm   # C4: F1@k + segment-count
ratio
    from backend.eval.score_calibration import fit_isotonic, fit_platt   # C2: calibrate cue
scores

    logger = logging.getLogger(__name__)

    # Where a comparison run appends one reproducible record. Tracked location (NOT under
    # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
    # regression.DEFAULT_BASELINE_PATH.
    DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
    _METRICS = ("precision", "recall", "f1", "mean_iou")


    class ExperimentError(ValueError):
        """A variant cannot be evaluated as configured (e.g. a learned variant asked to
        score an unlabeled video with no pinned model, or a gate referencing an absent
        feature)."""


```

```
    # ---------------------------------------------------------------------- Variant


    @dataclass
    class Variant:
        """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).

        A variant turns one video's persisted candidate windows + features into rally
        intervals. Every knob defaults to the **control** behaviour (no gate, no learned
        filter), so a variant that merely *carries* a new, disabled cue is byte-identical
        to control ? the core no-regression guarantee.

        Fields:
          - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
            for the leaderboard and for selecting the served path (the existing
            `segmenter_registry` + `IndexingConfig` do the actual selection in production).
            New cues are recorded here default-OFF.
          - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
            ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
            `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
            windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
          - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
            whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
            cue persists that feature).
          - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
            `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
            required for `compare_unlabeled` on a learned variant.
          - ``feature_names`` ? the persisted features the learned filter consumes. When set
            WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
            (honest) ? the recipe, not a pinned artifact.
          - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
            overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
        """

        name: str
        segmenter: str = "wasb_hybrid"
        config_overrides: Dict[str, Any] = field(default_factory=dict)
        cue_flags: Dict[str, bool] = field(default_factory=dict)
        min_crossings: int = 0
        min_players: int = 0
        classifier_model: Optional[str] = None
        feature_names: Optional[List[str]] = None
        threshold: float = 0.5
        merge_gap: float = 1.0
        # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
        # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
        # failure where a fixed 0.5 zeroed out a small-candidate video.
        tune_threshold: bool = False          # pick the F1-best threshold on the train fold
        calibrate: Optional[str] = None       # None | "platt" | "isotonic" ? calibrate
proba first

        def is_learned(self) -> bool:
            """True if this variant applies a learned classifier (pinned model or
recipe)."""
            return self.classifier_model is not None or bool(self.feature_names)

        def to_dict(self) -> dict:
            return {
                "name": self.name,
                "segmenter": self.segmenter,
                "config_overrides": self.config_overrides,
                "cue_flags": self.cue_flags,
```

```
                "min_crossings": self.min_crossings,
                "min_players": self.min_players,
                "classifier_model": self.classifier_model,
                "feature_names": self.feature_names,
                "threshold": self.threshold,
                "merge_gap": self.merge_gap,
                "tune_threshold": self.tune_threshold,
                "calibrate": self.calibrate,
            }

        @classmethod
        def from_dict(cls, d: dict) -> "Variant":
            known = {f for f in cls.__dataclass_fields__}  # type: ignore[attr-defined]
            return cls(**{k: v for k, v in d.items() if k in known})

        def spec_hash(self) -> str:
            """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
            and makes a result reproducible. The pinned **control** variant always hashes
the
            same, so a regression is always traceable to a recipe change, never to drift."""
            blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
            return sha1(blob.encode()).hexdigest()[:12]


    #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
    #: filter. Always the comparison reference; "rollback" = make this the served variant.
    CONTROL = Variant(name="control")


    # ---------------------------------------------------------------- persisted candidates


    @dataclass
    class VideoCandidates:
        """One video's persisted detection, ready for offline re-scoring.

        ``intervals`` are the candidate windows; ``features`` is column-major
        (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
        ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
        with `load_video_candidates`, or directly in tests.
        """

        name: str
        intervals: List[Interval]
        features: Dict[str, List[float]] = field(default_factory=dict)
        gts: Optional[List[Interval]] = None


    def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
        """True iff every candidate row carries the per-cue feature dicts the golden-feature
loaders
        read (``shuttle`` + ``person``; ``audio`` stays optional ? it is added later by
        ``fusion_audio.augment`` and older JSONs predate it).

        The velocity-windowing / distill corpus-growth path persists a LEANER
        ``*_golden_features.json`` row (``start``/``end`` + velocity stats, no cue dicts)
for clips
        that never went through fusion feature extraction. The loaders that build cue-rich
        ``VideoCandidates`` from these files ? ``ablation.load_videos``,
``fusion_compare.load_videos``
        and ``fusion_audio.load_videos_with_audio`` ? SKIP those rather than crash on
``c["shuttle"]``,
        so a MIXED corpus (some fusion files, some windowing-only) loads the fusion subset
cleanly.
        An empty candidate list is not a fusion file either.
```

```
192
193         Single source of truth for the three loaders (consolidated 2026-06-18 after PRs
#209/#210
194         each landed an independent copy)."""
195         return bool(candidates) and all("shuttle" in c and "person" in c for c in
candidates)
196
197
198     def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
199         """Persisted feature column, defaulting missing/short columns to 0.0 (matches
200         `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
201         col = vc.features.get(name)
202         n = len(vc.intervals)
203         if col is None:
204             logger.warning("variant references feature %r absent from %s ? treating as 0.0",
205                            name, vc.name)
206             return [0.0] * n
207         if len(col) != n:
208             raise ExperimentError(
209                 f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
210         return [float(x) for x in col]
211
212
213     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
214         cols = [_feature_column(vc, name) for name in feature_names]
215         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
216
217
218     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
219         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
players)."""
220         n = len(vc.intervals)
221         mask = [True] * n
222         if variant.min_crossings > 0:
223             col = _feature_column(vc, "net_crossings")
224             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
225         if variant.min_players > 0:
226             col = _feature_column(vc, "players_in_court")
227             mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
228         return mask
229
230
231     def predict(variant: Variant, vc: VideoCandidates,
232                 model: Optional[LogisticRegression] = None,
233                 threshold: Optional[float] = None,
234                 calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
235         """Variant prediction: gate by persisted features, optionally filter by a learned
236         model, then merge. Pure and deterministic.
237
238         ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
239         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
240         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
241         is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
242         fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
them.
243         """
244         mask = _gate_mask(variant, vc)
245         if variant.feature_names:
246             if model is None:
247                 if variant.classifier_model is None:
248                     raise ExperimentError(
249                         f"variant {variant.name!r} is learned but has no model to predict "
250                         f"with (pass a fitted model or set classifier_model)")
251                 model = LogisticRegression.load(variant.classifier_model)
```

```
            proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
            if calibrator is not None:
                proba = [calibrator(p) for p in proba]
            thr = variant.threshold if threshold is None else threshold
            mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
        kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
        return merge_overlaps(kept, gap_tol=variant.merge_gap)


    def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
                            train_videos: Sequence[VideoCandidates], iou: float
                            ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
        """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
        (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
        (use the variant default). Honest: never looks at the held-out video.
        """
        calibrator: Optional[Callable[[float], float]] = None
        if variant.calibrate:
            raw: List[float] = []
            y: List[int] = []
            for vc in train_videos:
                raw.extend(float(p) for p in
                           model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
                y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
            if variant.calibrate == "platt":
                calibrator = fit_platt(raw, y)
            elif variant.calibrate == "isotonic":
                calibrator = fit_isotonic(raw, y)
            else:
                raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
        threshold: Optional[float] = None
        if variant.tune_threshold:
            best_t, best_f1 = variant.threshold, -1.0
            for k in range(1, 20):                       # grid 0.05..0.95 on the train
fold's rally F1
                t = round(0.05 * k, 2)
                f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
                             vc.gts or [], iou).f1 for vc in train_videos]
                mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
                if mean_f1 > best_f1:
                    best_f1, best_t = mean_f1, t
            threshold = best_t
        return calibrator, threshold


    # ------------------------------------------------------------------- variant scoring


    def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
        """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
        evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`."""
        X: List[List[float]] = []
        y: List[int] = []
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
            X.extend(_feature_matrix(vc, variant.feature_names or []))
            y.extend(label_candidates(vc.intervals, vc.gts, iou))
```

```
        if not X:
            raise ExperimentError("cannot train a learned variant on zero candidates")
        return LogisticRegression().fit(X, y, variant.feature_names)


    def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
        """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
        mean aggregate.

        Prediction policy:
          - **static variant** (no learned filter): predict each video directly ? no
training,
            so no leakage; per-video scoring is already honest.
          - **learned, pinned model** (``classifier_model`` set): score every video with the
            SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
            report "how the shipped artifact does here", not for the promotion decision.
          - **learned recipe** (``feature_names``, no pinned model) + ``honest=True``: LOVO
?
            train on the OTHER videos, predict the held-out one. The number to trust.
          - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
        """
        for vc in videos:
            if vc.gts is None:
                raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")

        per_video: List[Dict[str, Any]] = []
        predictions: Dict[str, List[Interval]] = {}

        recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
        pinned_model = (LogisticRegression.load(variant.classifier_model)
                        if variant.classifier_model is not None else None)
        all_model = None
        if recipe_lovo and not honest:
            all_model = _train(variant, videos, iou)

        for i, vc in enumerate(videos):
            cal: Optional[Callable[[float], float]] = None
            thr: Optional[float] = None
            if recipe_lovo and honest:
                others = [v for j, v in enumerate(videos) if j != i]
                if not others:
                    raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
                model: Optional[LogisticRegression] = _train(variant, others, iou)
                if variant.tune_threshold or variant.calibrate:    # operating point from
the train fold
                    assert model is not None                       # _train never returns
None
                    cal, thr = _calibrate_and_tune(variant, model, others, iou)
            elif recipe_lovo:                       # honest=False ? one model over all
videos
                model = all_model
            else:                                   # static variant or pinned model
                model = pinned_model
            preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
            assert vc.gts is not None               # guarded at function top
            sc = score(preds, vc.gts, iou)
            per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
            predictions[vc.name] = preds

```

```
        aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
                     for m in _METRICS}
        return {
            "variant": variant.name,
            "spec_hash": variant.spec_hash(),
            "is_learned": variant.is_learned(),
            "honest_lovo": recipe_lovo and honest,
            "per_video": per_video,
            "aggregate": aggregate,
            "predictions": predictions,
        }


    def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
        """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
        return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}


    def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
        """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
        gts_by_name = {v.name: (v.gts or []) for v in videos}
        overlaps = _segm.DEFAULT_OVERLAPS
        f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
        ratios: List[float] = []
        for name, preds in eval_v.get("predictions", {}).items():
            gts = gts_by_name.get(name, [])
            got = _segm.f1_at_overlaps(preds, gts, overlaps)
            for ov in overlaps:
                f1k[ov].append(got[ov])
            ratios.append(_segm.segment_count_ratio(preds, gts))

        def _mean(xs: List[float]) -> float:
            return sum(xs) / len(xs) if xs else 0.0
        out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
        out["segment_count_ratio"] = _mean(ratios)
        return out


    def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
        """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).

        ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
        so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
        the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
        """
        a_f1 = [p["f1"] for p in eval_a["per_video"]]
        b_f1 = [p["f1"] for p in eval_b["per_video"]]
        return {
            "variant_a_ci": _ci_block(eval_a),
            "variant_b_ci": _ci_block(eval_b),
            "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
            "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
                             "variant_b": _segmentation_block(videos, eval_b)},
        }

```

```
420
421     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
422                 iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
423         """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
424         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
425
426         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
427         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
428         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
429
430         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
431         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
432         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
433         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
434         bare LOVO mean at n?6 is not trustworthy on its own.
435         """
436         if len(videos) < 1:
437             raise ExperimentError("compare needs at least one labeled video")
438         eval_a = evaluate_variant(videos, variant_a, iou, honest)
439         eval_b = evaluate_variant(videos, variant_b, iou, honest)
440
441         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
442         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
443
444         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
445         per_video_delta = [
446             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
447             for p in eval_b["per_video"]
448         ]
449
450         # One fingerprint over the whole label set ? detects "the deltas were measured
against
451         # different labels" the same way regression.gt_hash guards the no-regression
baseline.
452         all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
453
454         result: Dict[str, Any] = {
455             "iou": iou,
456             "label_set_hash": gt_hash(all_gts),
457             "num_videos": len(videos),
458             "variant_a": eval_a,
459             "variant_b": eval_b,
460             "delta": delta,
461             "delta_pct": delta_pct,
462             "per_video_delta": per_video_delta,
463         }
464         if honest_stats:
465             result["honest"] = honest_summary(videos, eval_a, eval_b)
466         return result
467
468
469     # ------------------------------------------------------------- SxS on unlabeled video
470
471
472     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
```

```
                        agree_iou: float = 0.5) -> Dict[str, Any]:
    """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).

    With no labels there is no lift to measure ? instead this surfaces where the two
    variants **agree vs diverge**, which is exactly what a human reviews (and what two
    highlight reels would show side by side):

      - ``agreed``  ? windows both variants found (matched at ``agree_iou``);
      - ``a_only``  ? A fired, B suppressed (B's added precision, if A was over-firing);
      - ``b_only``  ? B fired, A did not (B's new detections ? real recall or new FPs:
        the human / a later golden label adjudicates).

    Both variants must be predictable without labels: a learned variant therefore needs
a
    pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
    `metrics.match_intervals` ? no new matching logic.
    """
    preds_a = predict(variant_a, video)
    preds_b = predict(variant_b, video)
    mr = match_intervals(preds_a, preds_b, agree_iou)

    agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
              for m in mr.matches]
    agree_ious = [m.iou for m in mr.matches]
    a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
    b_only = [preds_b[i] for i in mr.unmatched_gt_indices]

    def _dur(ivs: List[Interval]) -> float:
        return sum(e - s for s, e in ivs)

    return {
        "video": video.name,
        "agree_iou": agree_iou,
        "variant_a": variant_a.name,
        "variant_b": variant_b.name,
        "num_a": len(preds_a),
        "num_b": len(preds_b),
        "num_agreed": len(agreed),
        "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
        "duration_a": _dur(preds_a),
        "duration_b": _dur(preds_b),
        "agreed": agreed,
        "a_only": a_only,
        "b_only": b_only,
    }


# ---------------------------------------------------------------------- leaderboard / DB


def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
                      timestamp: Optional[str] = None) -> Dict[str, Any]:
    """Append a compact, reproducible record of a `compare()` result to the JSON
    leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
    `regression_baseline.json`; deliberately the size of a baseline file, not a service
    (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
    """
    row: Dict[str, Any] = {
        "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
        "iou": result.get("iou"),
        "label_set_hash": result.get("label_set_hash"),
        "num_videos": result.get("num_videos"),
        "variant_a": {"name": result["variant_a"]["variant"],
                      "spec_hash": result["variant_a"]["spec_hash"],
                      "aggregate": result["variant_a"]["aggregate"]},
```

```
            "variant_b": {"name": result["variant_b"]["variant"],
                          "spec_hash": result["variant_b"]["spec_hash"],
                          "aggregate": result["variant_b"]["aggregate"]},
            "delta": result.get("delta"),
        }
        # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
the
        # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
        honest = result.get("honest") or {}
        if honest.get("paired_delta_f1"):
            row["honest_delta_f1"] = honest["paired_delta_f1"]
        entries: List[Dict[str, Any]] = []
        if os.path.isfile(path):
            with open(path, encoding="utf-8") as f:
                entries = json.load(f)
        entries.append(row)
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w", encoding="utf-8") as f:
            json.dump(entries, f, indent=2)
        logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
                    row["variant_a"]["name"], row["variant_b"]["name"],
                    (result.get("delta") or {}).get("f1", 0.0), path)
        return row


    def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
                              gts: Optional[List[Interval]] = None,
                              feature_names: Sequence[str] = ()) -> VideoCandidates:
        """Bridge persisted detection ? `VideoCandidates` via
`Database.query_candidate_segments`.

        ``feature_names`` selects which persisted features to lift out of each row's
``stats``
        JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
        function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
        but works against any `Database` (including an in-memory one in tests).
        """
        rows = db.query_candidate_segments(video_id, detector=detector)
        intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
        features: Dict[str, List[float]] = {}
        for name in feature_names:
            if name == "duration":
                features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
            else:
                features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
        return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
```

13. user (2026-06-18T07:48:05.848Z)

```
    """Regression runner ? verify the production tool against golden-set ground truth.

    Implements the explicit ask: *"given a video, get a golden set created and verify
    its precision and recall against the production tool."* This is deliberately a thin
    wrapper over machinery that already exists:

        ground truth  ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
        predictions   ? the production pipeline's stored DB rows     [backend.eval.harness]
        comparison    ? interval matching + P/R/F1                   [backend.eval.metrics]
```

```
        baseline      ? a per-video snapshot of "known good" numbers [this module]

    A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
    baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
    later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
    independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
    """
    import os
    import json
    import hashlib
    import logging
    from dataclasses import dataclass, asdict
    from typing import Dict, List, Optional

    from backend.infrastructure.database import Database
    from backend.eval import metrics
    from backend.eval.harness import get_predictions
    from backend.annotations import annotation_registry

    logger = logging.getLogger(__name__)

    # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
    # actually load a committed baseline to compare against.
    DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
    DEFAULT_TOLERANCE = 0.05


    def gt_hash(ground_truth: List[metrics.Interval]) -> str:
        """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
        norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
        return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]


    @dataclass
    class RegressionResult:
        video_id: str
        stage: str
        iou_threshold: float
        precision: float
        recall: float
        f1: float
        # Baseline values compared against (None when no baseline existed yet).
        baseline_precision: Optional[float]
        baseline_recall: Optional[float]
        tolerance: float
        regressed: bool
        reasons: List[str]
        # True only when a baseline existed to compare against. Distinguishes a genuine PASS
        # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
        has_baseline: bool = False
        gt_hash: Optional[str] = None

        def as_dict(self) -> dict:
            return asdict(self)


    # ---------------------------------------------------------------- baseline store

    def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
        """Load the per-video baseline snapshot file (returns {} if absent)."""
        if not os.path.isfile(path):
            return {}
        with open(path) as f:
```

```
            return json.load(f)


    def _baseline_key(video_id: str, stage: str) -> str:
        return f"{video_id}::{stage}"


    def save_baseline(video_id: str, stage: str, precision: float, recall: float,
                      f1: float, path: str = DEFAULT_BASELINE_PATH,
                      iou_threshold: Optional[float] = None, detector: Optional[str] = None,
                      gt_fingerprint: Optional[str] = None) -> None:
        """Snapshot a video/stage's current numbers as the new 'known good' baseline.

        Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
        later run computed under different settings (or against different labels) is
detected
        as incommensurable rather than silently compared.
        """
        baselines = load_baselines(path)
        baselines[_baseline_key(video_id, stage)] = {
            "video_id": video_id, "stage": stage,
            "precision": precision, "recall": recall, "f1": f1,
            "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
        }
        os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
        with open(path, "w") as f:
            json.dump(baselines, f, indent=2, sort_keys=True)
        logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
_baseline_key(video_id, stage), precision, recall)


    # ---------------------------------------------------------------- the runner

    def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
                stage: str = "final", iou_threshold: float = 0.5,
                detector: Optional[str] = None,
                tolerance: float = DEFAULT_TOLERANCE,
                baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
        """Score stored predictions for one video against ground truth and compare to
baseline.

        `ground_truth` is a list of (start, end) intervals ? typically obtained from an
        AnnotationProvider (see `regress_with_provider`). A regression is flagged when
        precision OR recall falls more than `tolerance` below the snapshotted baseline.
        If no baseline exists yet, `regressed` is False (nothing to compare to) and the
        caller can choose to snapshot the current numbers via `save_baseline`.
        """
        preds = get_predictions(db, video_id, stage, detector=detector)
        s = metrics.score(preds, ground_truth, iou_threshold)
        fp = gt_hash(ground_truth)

        baselines = load_baselines(baseline_path)
        b = baselines.get(_baseline_key(video_id, stage))

        reasons: List[str] = []
        regressed = False
        has_baseline = b is not None
        base_p = base_r = None
        if b is not None:
            base_p, base_r = b["precision"], b["recall"]
            # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
            # against different labels is not a valid reference ? flag it instead of
trusting it.
            b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
```

```
b.get("gt_hash")
            if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
                regressed = True
                reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
            if b_det is not None and b_det != detector:
                regressed = True
                reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
            if b_hash is not None and b_hash != fp:
                regressed = True
                reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
            if s.precision < base_p - tolerance:
                regressed = True
                reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
            if s.recall < base_r - tolerance:
                regressed = True
                reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
        else:
            reasons.append("no baseline recorded for this video/stage ? nothing to compare")

        return RegressionResult(
            video_id=video_id, stage=stage, iou_threshold=iou_threshold,
            precision=s.precision, recall=s.recall, f1=s.f1,
            baseline_precision=base_p, baseline_recall=base_r,
            tolerance=tolerance, regressed=regressed, reasons=reasons,
            has_baseline=has_baseline, gt_hash=fp,
        )


    def regress_with_provider(db: Database, video_id: str, video_ref: str,
                              tier: str = "file", stage: str = "final",
                              iou_threshold: float = 0.5,
                              detector: Optional[str] = None,
                              tolerance: float = DEFAULT_TOLERANCE,
                              baseline_path: str = DEFAULT_BASELINE_PATH,
                              guideline: Optional[str] = None,
                              sport: str = "badminton",
                              provider_kwargs: Optional[dict] = None) -> RegressionResult:
        """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.

        `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
        once those providers land). `video_ref` is whatever that provider expects (a CSV
        path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
        predictions in the DB (the file MD5).
        """
        provider = annotation_registry.get(tier, **(provider_kwargs or {}))
        gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
        return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
                       detector=detector, tolerance=tolerance, baseline_path=baseline_path)


    def format_result(r: RegressionResult) -> str:
        verdict = "REGRESSED" if r.regressed else "ok"
        head = (f"[{verdict}] {r.video_id} ({r.stage}) "
                f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
        if r.baseline_precision is not None:
            head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
        return head + "".join(f"\n   - {why}" for why in r.reasons)

```

14. assistant (2026-06-18T07:48:09.847Z)

Now let me examine a few areas more closely. Let me look at the Hungarian matcher's handling when scipy IS present but produces a wrong-cardinality result, and inspect the `eval_stats.paired_delta_ci` / `mean_with_ci` for n=0/1 behavior, plus the `merge_split_report` and `segment_count_ratio` for div-by-zero.

15. user (2026-06-18T07:48:10.739Z)

```
1       """Honest small-N evaluation statistics (course-corrections C1 + C3).
2
3       The ~6-match golden corpus + a regularised logistic arbiter is exactly the regime where
a
4       single leave-one-video-out *mean* lies: leave-one-out anti-correlates each fold's
training
5       and held-out label means at small N, and one lucky/unlucky fold swings the headline. The
6       ``compare``/LOVO machinery already produces **per-video held-out scores** ? this module
7       turns those into an honest summary:
8
9       - **C1 ? never a single mean.** ``mean_with_ci`` / ``bootstrap_ci`` report a confidence
10        interval; ``paired_sign_test`` and ``paired_delta_ci`` grade a B?A improvement by how
11        many *videos* it wins on (the distribution-free test the repo already leans on:
12        "sign test needs n ? ~n=10/9-wins for p<0.05"), so the promotion gate can't promote
noise.
13      - **C3 ? honest stacking.** ``out_of_fold_predictions`` / ``grouped_folds`` give a
learned
14        producer's *out-of-fold* outputs so feeding them to the arbiter doesn't leak (group by
15        match, never a held-out window from the same video).
16
17      See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C1+C3. All **additive**, pure (stdlib
only),
18      deterministic (bootstrap takes an explicit ``seed``) ? nothing here changes existing
19      ``compare``/``calibration.leave_one_video_out`` behaviour; callers opt in.
20      """
21      from __future__ import annotations
22
23      import math
24      import random
25      import statistics
26      from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple
27
28      Interval = Tuple[float, float]
29      FitPredict = Callable[[List[int], List[int]], List[Any]]
30
31
32      def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
33                       n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
34          """Percentile bootstrap CI for the mean of ``values``. Deterministic given ``seed``.
35
36          ``n<=1`` returns a degenerate ``(v, v)`` / ``(0, 0)`` interval ? there is no spread
to
37          resample. With n=6 (our corpus) this is the honest "how wide could the mean be?"
band.
38          """
39          vals = [float(v) for v in values]
40          n = len(vals)
41          if n == 0:
42              return (0.0, 0.0)
43          if n == 1:
44              return (vals[0], vals[0])
45          rng = random.Random(seed)
46          means: List[float] = []
47          for _ in range(max(1, n_boot)):
48              means.append(sum(vals[rng.randrange(n)] for _ in range(n)) / n)
49          means.sort()
```

```
        alpha = (1.0 - confidence) / 2.0
        last = len(means) - 1
        lo = means[max(0, int(math.floor(alpha * last)))]
        hi = means[min(last, int(math.ceil((1.0 - alpha) * last)))]
        return (lo, hi)


    def mean_with_ci(values: Sequence[float], confidence: float = 0.95,
                     n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
        """``{mean, std, n, ci_low, ci_high, confidence}`` ? report this, never a bare mean
(C1)."""
        vals = [float(v) for v in values]
        n = len(vals)
        mean = statistics.mean(vals) if vals else 0.0
        std = statistics.pstdev(vals) if n > 1 else 0.0
        lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
        return {"mean": mean, "std": std, "n": float(n),
                "ci_low": lo, "ci_high": hi, "confidence": confidence}


    def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
        """Two-sided exact sign test over per-fold deltas (B ? A).

        ``wins`` = folds where B beat A by more than ``eps``; ``losses`` the reverse; ties
        excluded. ``p_value`` is the exact two-sided binomial tail under p=0.5 ? the
        distribution-free "did B win on enough *videos*?" check.
        """
        wins = sum(1 for d in deltas if d > eps)
        losses = sum(1 for d in deltas if d < -eps)
        ties = len(deltas) - wins - losses
        n_eff = wins + losses
        if n_eff == 0:
            p = 1.0
        else:
            k = min(wins, losses)
            tail = sum(math.comb(n_eff, j) for j in range(k + 1)) * (0.5 ** n_eff)
            p = min(1.0, 2.0 * tail)
        return {"wins": float(wins), "losses": float(losses), "ties": float(ties),
                "n_effective": float(n_eff), "p_value": p}


    def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
                        confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
                        eps: float = 0.0) -> Dict[str, Any]:
        """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).

        ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
by
        fold (same video order). A promotion is honest when the delta CI clears 0 *and* the
sign
        test agrees ? not when a single mean ticked up.
        """
        n = min(len(a_values), len(b_values))
        deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
        summary = mean_with_ci(deltas, confidence, n_boot, seed)
        return {
            "mean_delta": summary["mean"],
            "ci_low": summary["ci_low"],
            "ci_high": summary["ci_high"],
            "n": float(n),
            "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
            "sign_test": paired_sign_test(deltas, eps),
        }


```

```
112     def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
113         """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest
unit.
114
115         The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
116         first-seen (deterministic). Groups with no complement (a single group) yield no
fold.
117         """
118         seen: List[Any] = []
119         for g in groups:
120             if g not in seen:
121                 seen.append(g)
122         n = len(groups)
123         folds: List[Tuple[List[int], List[int]]] = []
124         for g in seen:
125             test = [i for i in range(n) if groups[i] == g]
126             train = [i for i in range(n) if groups[i] != g]
127             if test and train:
128                 folds.append((train, test))
129         return folds
130
131
132     def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
List[Optional[Any]]:
133         """Out-of-fold predictions for honest stacking (C3).
134
135         For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on
the
136         other groups and returns predictions for ``test_idx`` (in that order); results are
137         scattered back to a list aligned to the input items. So a *learned* producer's
outputs,
138         when used as arbiter features, never include in-sample predictions ? the leakage
path
139         that inflates measured lift on a tiny corpus. Items in a singleton group stay
``None``
140         (needs >= 2 groups to predict).
141         """
142         n = len(groups)
143         oof: List[Optional[Any]] = [None] * n
144         for train_idx, test_idx in grouped_folds(groups):
145             preds = fit_predict(train_idx, test_idx)
146             for pos, ti in enumerate(test_idx):
147                 if pos < len(preds):
148                     oof[ti] = preds[pos]
149         return oof
150
```

16. user (2026-06-18T07:48:11.297Z)

```
1       """Segmentation-aware metrics (course-correction C4) ? make over-segmentation visible.
2
3       Temporal IoU (``metrics.score``) is nearly blind to *fragmentation*: a single rally
4       chopped into three predicted pieces, or two rallies merged across a dead-time gap, can
5       still post a moderate mean-IoU. The temporal-action-segmentation / -localization
6       literature grades those failure modes with **F1@k** (segmental F1 at several overlap
7       thresholds) and **mAP@tIoU** (ranked, confidence-aware average precision over a tIoU
8       sweep). Reporting these alongside temporal-IoU is what lets a reconciler's split/merge/
9       tighten win actually show up ? and stops an IoU-chasing rule from shredding rally
10      structure invisibly. See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C4.
11
12      This is **additive**: new functions over the existing ``metrics`` matcher; nothing here
13      changes ``score``/``compare``/LOVO behaviour. Pure (stdlib only), deterministic, no I/O
?
14      so it runs in the GPU-free / numpy-free lanes too.
15
```

```
    Conventions match ``metrics``: an ``Interval`` is a ``(start, end)`` tuple of seconds.
    A ``ScoredInterval`` is ``(interval, confidence)`` ? the confidence is whatever the
scorer
    emits per predicted rally (e.g. the classifier probability), used only to *rank* for AP.

    Note on the segmental *edit* score: the classic TAS edit/Levenshtein score operates on
the
    ordered sequence of segment *labels* and is degenerate for a single action class ?
rally/
    background segments strictly alternate, so the edit distance collapses to (twice) the
    segment-count difference, adding nothing over ``segment_count_ratio``. So we
intentionally
    omit it and instead make over-segmentation explicit with ``merge_split_report`` (a
bipartite
    overlap-graph breakdown into merge/split/matched/missed/spurious ? see below), alongside
    ``f1_at_overlaps`` (F1@k) + ``mean_average_precision`` (mAP@tIoU). Those, not a single-
class
    edit score, are the over-segmentation-sensitive metrics for this binary rally setting.
    """
    from __future__ import annotations

    from typing import Dict, List, Sequence, Tuple

    from backend.eval.metrics import Interval, interval_iou, score

    #: A predicted interval plus the confidence used to rank it for average precision.
    ScoredInterval = Tuple[Interval, float]

    #: Default overlap thresholds for F1@k ? the TAS convention F1@{10,25,50}.
    DEFAULT_OVERLAPS: Tuple[float, ...] = (0.1, 0.25, 0.5)
    #: Default tIoU sweep for mean average precision ? the action-detection convention.
    DEFAULT_TIOUS: Tuple[float, ...] = (0.3, 0.4, 0.5, 0.6, 0.7)


    def f1_at_overlaps(preds: List[Interval], gts: List[Interval],
                       overlaps: Sequence[float] = DEFAULT_OVERLAPS) -> Dict[float, float]:
        """Segmental F1 at each overlap threshold (``{overlap: f1}``) ? the TAS F1@k.

        Reuses the canonical greedy one-to-one matcher, so a fragmented prediction (3 pieces
        for 1 rally) scores extra false positives and a merged prediction misses a rally ?
        both drag F1 down where temporal-IoU alone would not flinch.
        """
        return {ov: score(preds, gts, iou_threshold=ov).f1 for ov in overlaps}


    def average_precision(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
                          iou_threshold: float = 0.5) -> float:
        """Average precision at a single tIoU (all-points / VOC-2010 interpolation).

        Predictions are ranked by descending confidence; each is a true positive iff it
        claims a still-unmatched ground-truth rally at IoU >= ``iou_threshold`` (highest
        available IoU wins, one GT per prediction). Returns AP in ``[0, 1]``. ``0.0`` when
        there are no ground-truth rallies or no predictions reach the threshold.
        """
        n_gt = len(gts)
        if n_gt == 0 or not scored_preds:
            return 0.0

        order = sorted(range(len(scored_preds)),
                       key=lambda i: (-scored_preds[i][1], i))   # conf desc, stable by
index
        matched_gt = set()
        precisions: List[float] = []
        recalls: List[float] = []
        cum_tp = 0
```

```
        cum_fp = 0
        for i in order:
            interval, _conf = scored_preds[i]
            best_iou, best_gi = 0.0, -1
            for gi, g in enumerate(gts):
                if gi in matched_gt:
                    continue
                iou = interval_iou(interval, g)
                if iou >= iou_threshold and iou > best_iou:
                    best_iou, best_gi = iou, gi
            if best_gi >= 0:
                matched_gt.add(best_gi)
                cum_tp += 1
            else:
                cum_fp += 1
            precisions.append(cum_tp / (cum_tp + cum_fp))
            recalls.append(cum_tp / n_gt)

        # Monotone precision envelope (take the max precision to the right), then integrate
        # over recall steps: AP = sum_k (recall_k - recall_{k-1}) * precision_env_k.
        for i in range(len(precisions) - 2, -1, -1):
            precisions[i] = max(precisions[i], precisions[i + 1])
        ap = 0.0
        prev_recall = 0.0
        for p, r in zip(precisions, recalls):
            if r > prev_recall:
                ap += p * (r - prev_recall)
                prev_recall = r
        return ap


    def mean_average_precision(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
                               iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> float:
        """Mean of ``average_precision`` over a tIoU sweep (the standard mAP@tIoU)."""
        tious = list(iou_thresholds)
        if not tious:
            return 0.0
        return sum(average_precision(scored_preds, gts, t) for t in tious) / len(tious)


    def segment_count_ratio(preds: List[Interval], gts: List[Interval]) -> float:
        """Predicted-to-GT segment-count ratio ? a crude fragmentation diagnostic.

        ``> 1`` hints over-segmentation (more predicted pieces than rallies), ``< 1`` hints
        merging/misses. Only meaningful with ground truth present; returns ``0.0`` if
``gts``
        is empty (use F1@k / mAP for the graded picture).
        """
        if not gts:
            return 0.0
        return len(preds) / len(gts)


    def _overlaps(a: Interval, b: Interval) -> bool:
        """True iff two intervals share any positive temporal extent."""
        return min(a[1], b[1]) - max(a[0], b[0]) > 0.0


    def merge_split_report(preds: List[Interval], gts: List[Interval]) -> Dict[str, float]:
        """Explicit over-/under-segmentation breakdown via the bipartite overlap graph.

        Where ``segment_count_ratio`` only compares *counts* and ``f1_at_overlaps`` blends
merge
        and split into one F1, this names the two failure modes directly ? the diagnostic
that
```

```
        surfaced our local-segmenter over-MERGE (one giant window swallowing many rallies).
Build
        the bipartite graph (edge = any positive temporal overlap between a pred and a gt),
take its
        connected components, and classify each by ``(#preds, #gts)``:

          - ``(1,1)`` matched   ``(1,0)`` spurious (pred, no gt)   ``(0,1)`` missed (gt, no
pred)
          - ``(1,>1)`` **merge** ? one predicted window covers ?2 rallies (over-MERGE; our
failure)
          - ``(>1,1)`` **split** ? one rally cut across ?2 predictions (over-segmentation)
          - ``(>1,>1)`` complex (tangled many-to-many)

        Returns counts plus ``merge_rate`` (fraction of GT rallies swallowed inside a merge
group)
        and ``split_rate`` (fraction of GT rallies cut by ?2 predictions) ? the two headline
        over-segmentation rates. Pure, deterministic, stdlib-only.
        """
        n, m = len(preds), len(gts)
        p_adj: List[List[int]] = [[] for _ in range(n)]
        g_adj: List[List[int]] = [[] for _ in range(m)]
        for i, p in enumerate(preds):
            for j, g in enumerate(gts):
                if _overlaps(p, g):
                    p_adj[i].append(j)
                    g_adj[j].append(i)

        seen_p: set = set()
        seen_g: set = set()
        comps: List[Tuple[set, set]] = []
        for start in range(n):
            if start in seen_p:
                continue
            ps: set = set()
            gs: set = set()
            stack: List[Tuple[str, int]] = [("p", start)]
            while stack:
                side, k = stack.pop()
                if side == "p":
                    if k in ps:
                        continue
                    ps.add(k)
                    seen_p.add(k)
                    stack.extend(("g", j) for j in p_adj[k] if j not in gs)
                else:
                    if k in gs:
                        continue
                    gs.add(k)
                    seen_g.add(k)
                    stack.extend(("p", i) for i in g_adj[k] if i not in ps)
            comps.append((ps, gs))
        comps.extend((set(), {j}) for j in range(m) if j not in seen_g)  # gts with no
overlapping pred

        counts = {"matched": 0, "merges": 0, "splits": 0, "spurious": 0, "missed": 0,
"complex": 0}
        gts_merged = gts_split = 0
        for ps, gs in comps:
            a, b = len(ps), len(gs)
            if (a, b) == (1, 1):
                counts["matched"] += 1
            elif (a, b) == (1, 0):
                counts["spurious"] += 1
            elif (a, b) == (0, 1):
                counts["missed"] += 1
```

```
194             elif a == 1 and b > 1:
195                 counts["merges"] += 1
196                 gts_merged += b
197             elif b == 1 and a > 1:
198                 counts["splits"] += 1
199                 gts_split += 1
200             else:
201                 counts["complex"] += 1
202         out: Dict[str, float] = {k: float(v) for k, v in counts.items()}
203         out["merge_rate"] = (gts_merged / m) if m else 0.0
204         out["split_rate"] = (gts_split / m) if m else 0.0
205         return out
206 
207 
208     def segmentation_report(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
209                             overlaps: Sequence[float] = DEFAULT_OVERLAPS,
210                             iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> Dict[str,
object]:
211         """One-call bundle: F1@k + mAP@tIoU + per-tIoU AP + count ratio + merge/split
breakdown.
212 
213         ``scored_preds`` carry confidences for AP; F1@k + the merge/split breakdown use the
bare
214         intervals. Returns a plain dict (JSON-friendly, string keys) so it can ride next to
a
215         ``compare`` result or land in the experiment leaderboard.
216         """
217         preds = [iv for iv, _conf in scored_preds]
218         return {
219             "f1_at_overlaps": {f"{ov:g}": v for ov, v in f1_at_overlaps(preds, gts,
overlaps).items()},
220             "ap_at_tiou": {f"{t:g}": average_precision(scored_preds, gts, t) for t in
iou_thresholds},
221             "mean_ap": mean_average_precision(scored_preds, gts, iou_thresholds),
222             "segment_count_ratio": segment_count_ratio(preds, gts),
223             "merge_split": merge_split_report(preds, gts),
224             "num_pred": len(preds),
225             "num_gt": len(gts),
226         }
227 
```

17. assistant (2026-06-18T07:48:17.729Z)

I found a real bug in `over_seg_guard`. Let me verify by examining the `aggregate` function path and confirming what `score_intervals` rows contain. The `over_seg_guard` in `compare` (rally_seg_eval.py) is called with the rows from `score_intervals`, which contain `split_count` and `merge_count` but NOT `merge_rate`/`split_rate` as those keys. Let me check.

Looking at `score_intervals` return dict (lines 64-78): it includes `merge_rate`, `split_rate`, `merges`, `splits`, `missed`, `spurious`, AND `split_count`, `merge_count`. So `merge_rate`/`split_rate` ARE present. Good ? that path is fine.

Now let me check the `over_seg_guard` `_aligned` function for a subtle bug, and the boundary error report `max(..., default=None)` with a lambda. Let me also look at the `_hungarian_match_1to1` cardinality issue more carefully. Let me check the test file for tolerance_metrics to understand the contract.

18. user (2026-06-18T07:48:18.281Z)

tests\test_tolerance_metrics.py

19. user (2026-06-18T07:48:18.478Z)

No files found

20. user (2026-06-18T07:48:21.782Z)

```
1       """Tests for the R2 tolerance-match metric (backend/eval/tolerance_metrics.py).
2
3       Pure/offline. The match cases are deliberately UNAMBIGUOUS so the greedy fallback and
the exact
4       Hungarian path agree ? results are environment-independent (CI may lack scipy)."""
5       import pytest
6
7       from backend.eval import tolerance_metrics as tm
8
9
10      def test_exact_match_is_perfect():
11          gts = [(0.0, 5.0), (10.0, 16.0), (20.0, 27.0)]
12          matches, p, r, f1 = tm.match_1to1(gts, gts, tau_start=2.0, tau_end=1.5)
13          assert len(matches) == 3
14          assert p == 1.0 and r == 1.0 and f1 == 1.0
15          assert all(ds == 0.0 and de == 0.0 for _, _, ds, de in matches)
16
17
18      def test_start_fuzzy_within_tau_still_matches():
19          # start shifted +1.5s (? ?_start 2.0), end shifted +1.0s (? ?_end 1.5) ? matches.
20          gts = [(0.0, 5.0)]
21          preds = [(1.5, 6.0)]
22          matches, p, r, f1 = tm.match_1to1(preds, gts, 2.0, 1.5)
23          assert len(matches) == 1 and f1 == 1.0
24          assert matches[0][2] == pytest.approx(1.5) and matches[0][3] == pytest.approx(1.0)
25
26
27      def test_end_loose_beyond_tau_does_not_match():
28          # start exact, end shifted +2.0s (> ?_end 1.5) ? NOT a match (the rally-end deficit
is exposed).
29          gts = [(0.0, 5.0)]
30          preds = [(0.0, 7.0)]
31          matches, p, r, f1 = tm.match_1to1(preds, gts, 2.0, 1.5)
32          assert matches == [] and f1 == 0.0
33          # but it WOULD match at a looser ?_end
34          assert len(tm.match_1to1(preds, gts, 2.0, 2.0)[0]) == 1
35
36
37      def test_over_production_costs_precision():
38          # 3 preds for 1 rally: only one can match 1:1 ? R=1.0 but P=1/3 (over-production
penalised).
39          gts = [(10.0, 16.0)]
40          preds = [(10.0, 16.0), (10.1, 16.1), (10.2, 16.2)]
41          matches, p, r, f1 = tm.match_1to1(preds, gts, 2.0, 1.5)
42          assert len(matches) == 1
43          assert r == 1.0 and p == pytest.approx(1 / 3)
44
45
46      def test_one_to_one_no_double_assignment():
47          # 2 preds, 2 gts, both eligible to the closer gt ? strict 1:1 must use each once.
48          gts = [(0.0, 5.0), (6.0, 11.0)]
49          preds = [(0.2, 5.2), (6.1, 11.1)]
50          matches, p, r, f1 = tm.match_1to1(preds, gts, 2.0, 1.5)
51          assert len(matches) == 2 and p == 1.0 and r == 1.0
52          assert sorted(j for _, j, _, _ in matches) == [0, 1]
53
54
55      def test_empty_inputs():
56          assert tm.match_1to1([], [(0.0, 5.0)], 2.0, 1.5) == ([], 0.0, 0.0, 0.0)
57          assert tm.match_1to1([(0.0, 5.0)], [], 2.0, 1.5) == ([], 0.0, 0.0, 0.0)
58          assert tm.match_1to1([], [], 2.0, 1.5) == ([], 0.0, 0.0, 0.0)
59
60
```

```
    def test_greedy_fallback_directly_matches_high_level():
        # the scipy-free path must give the same cardinality on an unambiguous case
        gts = [(0.0, 5.0), (10.0, 15.0)]
        preds = [(0.5, 5.5), (10.5, 15.5)]
        g = tm._greedy_match_1to1(preds, gts, 2.0, 1.5)
        assert len(g) == 2 and sorted(j for _, j, _, _ in g) == [0, 1]


    def test_boundary_error_report_splits_start_end():
        # unconditional best-overlap: gt0 best-overlaps pred0 (?start +1, ?end +3);
        # gt1 best-overlaps pred1 (?start -1, ?end +2). |?start| {1,1}, |?end| {3,2}.
        gts = [(0.0, 10.0), (20.0, 30.0)]
        preds = [(1.0, 13.0), (19.0, 32.0)]
        rep = tm.boundary_error_report(preds, gts)
        assert rep["dstart"]["abs_median"] == pytest.approx(1.0)
        assert rep["dend"]["abs_median"] == pytest.approx(2.5)
        assert rep["dstart"]["signed_median"] == pytest.approx(0.0)  # +1, -1
        assert rep["dstart"]["n"] == 2.0


    def test_boundary_error_unconditional_surfaces_loose_end():
        # a pred whose end is far beyond ? (won't match in 1:1) STILL contributes its loose
end here
        # ? this is the survivorship-bias fix: the end-deficit must be visible, not hidden
as a miss.
        gts = [(0.0, 5.0)]
        preds = [(0.0, 9.0)]  # ?end +4.0, way beyond ?_end
        rep = tm.boundary_error_report(preds, gts)
        assert rep["dend"]["abs_median"] == pytest.approx(4.0)
        # and it does NOT 1:1-match at ?_end=1.5 (deficit shows as recall=0 there)
        assert tm.match_1to1(preds, gts, 2.0, 1.5)[0] == []


    def test_over_seg_report_flags_merge_and_split():
        # merge: one pred covers two rallies
        merge = tm.over_seg_report([(0.0, 30.0)], [(2.0, 8.0), (15.0, 22.0)])
        assert merge["merge_count"] == 1 and merge["split_count"] == 0
        # split: two preds cut one rally
        split = tm.over_seg_report([(0.0, 4.0), (5.0, 9.0)], [(0.0, 9.0)])
        assert split["split_count"] == 1 and split["merge_count"] == 0


    def test_tolerance_report_block_shape():
        gts = [(0.0, 5.0), (10.0, 16.0)]
        preds = [(0.3, 5.3), (10.0, 18.0)]  # #1 matches; #2 end too loose at ?_end=1.5
        rep = tm.tolerance_report(preds, gts, 2.0, 1.5)
        assert rep["n_pred"] == 2 and rep["n_gt"] == 2 and rep["matched"] == 1
        assert rep["recall"] == pytest.approx(0.5) and rep["precision"] ==
pytest.approx(0.5)
        assert set(rep["boundary_error"]) == {"dstart", "dend"}
        assert set(rep["over_seg"]) == {"split_count", "merge_count"}


    def test_tolerance_sweep_monotonic_nondecreasing():
        gts = [(0.0, 5.0), (10.0, 16.0), (20.0, 27.0)]
        preds = [(0.5, 6.0), (10.5, 17.5), (20.5, 29.0)]  # increasingly loose ends
        sweep = tm.tolerance_sweep(preds, gts)
        vals = [sweep[t] for t in tm.SWEEP_TAUS]
        assert vals == sorted(vals)  # F1@? never decreases as ? grows


    # ---------------------------------------------------------------------- #
    # R1 ? net over-segmentation promotion guard
    # ---------------------------------------------------------------------- #
    def _row(name, sc, mc, sr, mr):
```

```
        return {"name": name, "split_count": sc, "merge_count": mc, "split_rate": sr,
"merge_rate": mr}


    def test_over_seg_guard_passes_when_net_cost_falls():
        # Candidate trades a baseline merge (rate 1.0) for a few splits (rate 0.2) ? net
cost down.
        base = [_row("v1", 0, 1, 0.0, 1.0)]
        cand = [_row("v1", 3, 0, 0.2, 0.0)]
        g = tm.over_seg_guard(base, cand)  # rate-based, w_merge=2, w_split=1
        assert g["passes"] is True
        assert g["base_net"] == pytest.approx(2.0)   # 2*1.0 + 1*0.0
        assert g["cand_net"] == pytest.approx(0.2)   # 2*0.0 + 1*0.2
        assert g["net_delta"] < 0


    def test_over_seg_guard_strict_blocks_what_net_passes():
        # The structural mega?bounded case: split_count rises 0?3 (strict trips) yet
merge_rate falls.
        base = [_row("v1", 0, 1, 0.0, 1.0)]
        cand = [_row("v1", 3, 0, 0.2, 0.0)]
        g = tm.over_seg_guard(base, cand)
        assert g["strict_passes"] is False   # raw split COUNT increased
        assert g["passes"] is True           # but the NET rule promotes it
        assert len(g["new_splits"]) == 1 and g["new_splits"][0]["name"] == "v1"


    def test_over_seg_guard_blocks_real_merge_rate_regression():
        # A genuine over-merge regression: merge_rate RISES on a video ? the categorical
block fires,
        # even if the weighted net cost happened to fall elsewhere.
        base = [_row("v1", 0, 0, 0.0, 0.1), _row("v2", 5, 0, 0.5, 0.0)]
        cand = [_row("v1", 0, 1, 0.0, 0.6), _row("v2", 0, 0, 0.0, 0.0)]
        g = tm.over_seg_guard(base, cand)
        assert g["no_merge_rate_regress"] is False
        assert g["passes"] is False
        assert g["merge_rate_regress"][0]["name"] == "v1"


    def test_over_seg_guard_count_basis_can_flip_verdict():
        # raw COUNT basis is weight-sensitive where the RATE basis is robust ? the reason
rates are the
        # default. One mega-window (count 1, rate 1.0) ? 9 two-glue windows (count 9, rate
0.5).
        base = [_row("v1", 0, 1, 0.0, 1.0)]
        cand = [_row("v1", 4, 9, 0.1, 0.5)]
        rate = tm.over_seg_guard(base, cand, use_rates=True)
        count = tm.over_seg_guard(base, cand, use_rates=False)
        assert rate["passes"] is True            # fewer rallies hidden ? promote
        assert count["passes"] is False          # raw merge count 1?9 ? net count rises ?
block
        assert rate["no_merge_rate_regress"] is True


    def test_over_seg_guard_weights_are_tunable_and_reported():
        base = [_row("v1", 0, 1, 0.0, 1.0)]
        cand = [_row("v1", 3, 0, 0.6, 0.0)]
        # default w_merge=2 passes (2.0 ? 0.6); flat w_merge=1 still passes here (1.0 ? 0.6)
        assert tm.over_seg_guard(base, cand, weight_merge=1.0)["passes"] is True
        g = tm.over_seg_guard(base, cand, weight_merge=3.0, weight_split=1.0)
        assert g["weight_merge"] == 3.0 and g["weight_split"] == 1.0


    def test_over_seg_guard_aligns_by_name_not_order():
        base = [_row("a", 0, 1, 0.0, 1.0), _row("b", 0, 1, 0.0, 1.0)]
```

```
181         cand = [_row("b", 1, 0, 0.1, 0.0), _row("a", 1, 0, 0.1, 0.0)]  # reversed order
182         g = tm.over_seg_guard(base, cand)
183         assert g["n"] == 2 and g["passes"] is True and not g["merge_rate_regress"]
184
185
186     def test_over_seg_guard_empty_is_trivially_pass():
187         g = tm.over_seg_guard([], [])
188         assert g["passes"] is True and g["n"] == 0 and g["net_delta"] == 0.0
189
```

21. assistant (2026-06-18T07:48:32.331Z)

The tests confirm the contracts. Now let me focus on a subtle correctness concern in the Hungarian matcher and the `_aligned` partial-overlap case in `over_seg_guard`. Let me also examine the served_gate_a `aggregate` n=0 path and `score`/`Score` for `seg_ratio` when gts empty.

Let me look closely at the Hungarian matcher's potential cardinality mismatch with greedy. The docstring claims they "agree on near-diagonal eligibility graphs", but the Hungarian path minimizes *total cost*, not *cardinality*. With `linear_sum_assignment` over a square cost matrix padded with `big`, scipy minimizes total cost. There's a known issue: minimizing total cost does NOT guarantee maximum cardinality when ineligible cells have a finite large cost. Let me think through whether this can drop a valid match.

Consider: cost matrix where eligible cells = small costs, ineligible = `big` (1e6). `linear_sum_assignment` minimizes the SUM. With `big=1e6`, choosing an ineligible cell costs 1e6, while any eligible assignment costs ? ~ small. So the solver will always prefer to make eligible assignments where possible ? but it could trade off: it might use an eligible cell sub-optimally. Actually, since `big` (1e6) is vastly larger than any real eligible cost (bounded by `tau_start+tau_end ? 3.5`), the solver always maximizes the number of eligible assignments first (each eligible assignment saves ~1e6). So cardinality IS maximized. That's fine.

Let me check `metrics.score` and the `Score` namedtuple for the seg_ratio / n=0 edge in served_gate_a, and confirm `merges`/`splits` keys exist where `over_seg_report` reads `r["splits"]`/`r["merges"]`.

22. user (2026-06-18T07:48:33.257Z)

```
1       """Pure temporal interval-matching metrics for rally evaluation.
2
3       No I/O, no DB, no global state ? every function here is deterministic and unit
4       tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5       start < end. We score a list of *predicted* rally intervals against a list of
6       *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7       """
8
9       from dataclasses import dataclass, field
10      from typing import List, Tuple, Optional
11
12      Interval = Tuple[float, float]
13
14
15      def interval_iou(a: Interval, b: Interval) -> float:
16          """Temporal intersection-over-union of two intervals.
17
18          Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19          overlap / union, both measured as durations on the timeline.
20          """
21          a_start, a_end = a
22          b_start, b_end = b
23          if a_end <= a_start or b_end <= b_start:
24              return 0.0
25          inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26          if inter <= 0.0:
27              return 0.0
```

```
        union = (a_end - a_start) + (b_end - b_start) - inter
        return inter / union if union > 0 else 0.0


    @dataclass
    class Match:
        """One matched predicted?ground-truth pair."""
        pred_index: int
        gt_index: int
        iou: float


    @dataclass
    class MatchResult:
        """Outcome of matching predictions to ground truth at a given IoU threshold."""
        matches: List[Match] = field(default_factory=list)
        unmatched_pred_indices: List[int] = field(default_factory=list)  # false positives
        unmatched_gt_indices: List[int] = field(default_factory=list)    # false negatives
(misses)


    def match_intervals(preds: List[Interval], gts: List[Interval],
                        iou_threshold: float = 0.5) -> MatchResult:
        """Greedy one-to-one matching of predictions to ground truth by descending IoU.

        Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
        are claimed highest-IoU first, and each prediction/GT can be used once. This
        keeps a single prediction from being credited for two ground-truth rallies
        (and vice versa).
        """
        candidates = []
        for pi, p in enumerate(preds):
            for gi, g in enumerate(gts):
                iou = interval_iou(p, g)
                # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
would
                # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
                if iou > 0.0 and iou >= iou_threshold:
                    candidates.append((iou, pi, gi))
        # Highest IoU first; ties broken by indices for determinism.
        candidates.sort(key=lambda c: (-c[0], c[1], c[2]))

        used_pred, used_gt = set(), set()
        matches: List[Match] = []
        for iou, pi, gi in candidates:
            if pi in used_pred or gi in used_gt:
                continue
            used_pred.add(pi)
            used_gt.add(gi)
            matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))

        unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
        unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
        return MatchResult(matches=matches,
                           unmatched_pred_indices=unmatched_pred,
                           unmatched_gt_indices=unmatched_gt)


    @dataclass
    class Score:
        """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
        iou_threshold: float
        num_pred: int
        num_gt: int
        true_positives: int
```

```
91          false_positives: int
92          false_negatives: int
93          precision: float
94          recall: float
95          f1: float
96          mean_iou: float            # over matched pairs (0.0 if none)
97          mean_start_error: float    # mean |pred_start - gt_start| over matches, seconds
98          mean_end_error: float      # mean |pred_end - gt_end| over matches, seconds
99
100         def as_dict(self) -> dict:
101             return self.__dict__.copy()
102
103
104     def score(preds: List[Interval], gts: List[Interval],
105               iou_threshold: float = 0.5,
106               match_result: Optional[MatchResult] = None) -> Score:
107         """Compute precision/recall/F1, mean matched IoU, and boundary errors.
108
109         Pass a precomputed `match_result` to avoid re-matching (e.g. when also
110         extracting the failure lists); otherwise matching is done here.
111         """
112         mr = match_result if match_result is not None else match_intervals(preds, gts,
iou_threshold)
113
114         tp = len(mr.matches)
115         fp = len(mr.unmatched_pred_indices)
116         fn = len(mr.unmatched_gt_indices)
117
118         precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
119         recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
120         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
else 0.0
121
122         if mr.matches:
123             mean_iou = sum(m.iou for m in mr.matches) / tp
124             mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
mr.matches) / tp
125             mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
mr.matches) / tp
126         else:
127             mean_iou = mean_start_err = mean_end_err = 0.0
128
129         return Score(
130             iou_threshold=iou_threshold,
131             num_pred=len(preds), num_gt=len(gts),
132             true_positives=tp, false_positives=fp, false_negatives=fn,
133             precision=precision, recall=recall, f1=f1,
134             mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
135         )
136
137
138     def pr_curve(preds: List[Interval], gts: List[Interval],
139                  thresholds: Optional[List[float]] = None) -> List[Score]:
140         """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
141
142         Lets you see strict-vs-lenient behaviour rather than a single operating point.
143         """
144         if thresholds is None:
145             thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
146         return [score(preds, gts, iou_threshold=t) for t in thresholds]
147
```

23. assistant (2026-06-18T07:48:42.291Z)

Now let me revisit the `over_seg_guard` `_aligned` function carefully for a real bug. When

`base`/`cand` have `name` keys, it aligns by name. But the per-video regression loop (lines 228-237) iterates over `pairs` from `_aligned()`. The corpus-mean costs (`base_costs`/`cand_costs`) are computed over `pairs` too. This is consistent.

However, there's a subtle issue: in `compare()` (rally_seg_eval.py line 190), `over_seg_guard` is called with `[rb for rb, _ in paired]` and `[rc for _, rc in paired]`. These rows DO have `name` keys (set in `evaluate`). The `paired` list is already name-matched, so positions align. The guard then re-aligns by name ? fine.

Let me now look at a potential real issue: the `_aligned()` name path uses `cmap[b["name"]]` which would `KeyError` if there are duplicate handling... no, it filters `if b["name"] in cmap`. Fine.

Let me focus on `boundary_error_report`'s `max(preds, key=..., default=None)`. When `preds` is empty, `max` with `default=None` returns None ? handled. Good.

Now let me check a real concern in `windows_for` (rally_seg_eval.py) vs `gate_windows` signature, and whether `score_intervals` `merge_rate`/`split_rate` keys propagate properly. Let me verify the `rally_gate.gate_windows` contract and the `GateWindow.kept` attribute.

24. user (2026-06-18T07:48:43.527Z)

```
1       """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3       WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4       the opponent for service after losing a point, that single trajectory looks like a
5       rally. The discriminator is structural, not duration-based: a real rally has the
6       shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7       single service feed crosses ~once.
8
9       This module is camera-agnostic and needs NO new model ? it works on the trajectory
10      we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11      axis with the larger shuttle spread) and the net position (median shuttle coord on
12      that axis ? the shuttle clusters at the two ends and crosses the net region often,
13      so the median sits near the net), then counts sign changes of (coord - net) across
14      the visible points inside a candidate window, with a deadband to ignore jitter.
15
16      Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17      Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18      windows from trajectory_to_action_windows, or to fold into it later.
19      """
20      from __future__ import annotations
21
22      from dataclasses import dataclass
23      from typing import List, Optional, Sequence, Tuple
24
25      Interval = Tuple[float, float]
26
27
28      @dataclass
29      class GatedWindow:
30          start: float
31          end: float
32          crossings: int
33          visible_points: int
34          kept: bool
35
36
37      def _spread(vals: Sequence[float]) -> float:
38          """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39          if len(vals) < 2:
40              return 0.0
41          s = sorted(vals)
42          lo = s[max(0, int(0.05 * (len(s) - 1)))]
43          hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
```

```
        return hi - lo


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])


    def estimate_play_axis(points) -> Tuple[str, float, float]:
        """Return (axis 'x'|'y', net_value, span) from visible points.

        Play axis = the image axis along which the shuttle travels most (larger spread).
        For a baseline camera that's Y (players top/bottom); for a side camera, X.
        """
        xs = [p.x for p in points if p.visible]
        ys = [p.y for p in points if p.visible]
        sx, sy = _spread(xs), _spread(ys)
        if sx >= sy:
            return "x", _median(xs), sx
        return "y", _median(ys), sy


    def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
        """Count sign changes of (coord - net) across visible points, ignoring the
        deadband zone near the net (so jitter at the net doesn't inflate the count)."""
        crossings = 0
        last_side = None
        for p in window_points:
            if not p.visible:
                continue
            v = (p.x if axis == "x" else p.y) - net
            if abs(v) < deadband:
                continue
            side = v > 0
            if last_side is not None and side != last_side:
                crossings += 1
            last_side = side
        return crossings


    def _points_in(points, fps: float, start: float, end: float):
        return [p for p in points if start * fps <= p.frame <= end * fps]


    def gate_windows(points, windows: List[Interval], fps: float,
                     min_crossings: int = 2, deadband_frac: float = 0.06,
                     estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
        """Annotate each (start,end) window with its shuttle net-crossing count and a
        keep decision (crossings >= min_crossings). Deadband is a fraction of the play
        axis span. Net/axis are estimated once over the whole trajectory (stable).

        `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
        `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
        per window ? same result, avoids the per-window re-estimation."""
        axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
        deadband = deadband_frac * span
        out: List[GatedWindow] = []
        for (s, e) in windows:
            wp = _points_in(points, fps, s, e)
            c = count_net_crossings(wp, axis, net, deadband)
```

```
107             vis = sum(1 for p in wp if p.visible)
108             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                                    kept=c >= min_crossings))
110         return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                    min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118
```

25. assistant (2026-06-18T07:48:55.185Z)

`windows_for` calls `gate_windows(points, ivs, video["fps"], min_crossings=min_crossings)` ? signature matches. Good.

Now let me focus on the one genuine logic concern I've spotted. In `over_seg_guard`, the `_aligned()` name path returns ONLY videos present in both base and cand (`if b["name"] in cmap`). But the per-video regression loop and the net-cost are both computed over those `pairs`, so `n` reflects the intersection. That's internally consistent ? but if `cand` is MISSING a video that `base` has, the guard silently drops it from the net-cost average and won't catch a regression on the dropped video. This is more of a design choice than a bug, and in the actual `compare()` caller the rows always align. Not a reportable defect.

Let me check one more genuinely suspicious thing: the `tolerance_sweep` "monotonic non-decreasing" claim and whether the Hungarian/greedy can violate it. But that's a property claim, not a crash. Let me instead verify the served_gate_a aggregate with n=0 ? `_mean` returns 0.0 when n==0, fine; `wins`/`losses` fine.

Let me look at whether `_percentile` in tolerance_metrics has an off-by-one with `int(k)` for negative ? p is always 25/75, xs non-negative, fine.

Now let me look at a potential real bug: in `compare()` in rally_seg_eval.py, the `over_seg_guard` is passed rows that have `merge_rate`/`split_rate` from `score_intervals`. But wait ? `score_intervals` computes `merge_rate`/`split_rate` from `merge_split_report(preds, gts)` which returns `(gts_merged / m)` where `m = len(gts)`. Since `evaluate` skips videos with no gts (`if not v["gts"]`), `m >= 1`, so no div-by-zero there.

Let me check the `IndexingConfig` defaults referenced in windowing.py to ensure those attributes exist (a missing attr would crash at import of `SERVED`).

26. user (2026-06-18T07:48:57.254Z)

```
139:    max_window_duration: float = 6.0
143:    # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
145:    window_hard_cap: bool = False
149:    # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only cuts.
153:    window_inactivity_end: float = 1.5
154:    inactivity_rally_thresh: float = 0.20
155:    inactivity_min_density: float = 0.30
158:    # than window_blob_factor × max_window_duration (no internal gap, no rally-end signal ?
e.g.
161:    # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
164:    window_blob_fallback: bool = False
165:    # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
169:    window_blob_factor: float = 4.0
```

27. assistant (2026-06-18T07:49:01.984Z)

Interesting ? note `max_window_duration` defaults to `6.0` (not 0) and `window_inactivity_end` defaults to `1.5` (not 0). The windowing.py docstrings say "default 0=OFF" but the actual config default is non-zero. That's a docstring/comment staleness issue, not a runtime defect ? the code reads the actual value via `float(idx.max_window_duration)`. Not reportable as a bug (pure comment drift).

Let me verify `idx.wasb.velocity_threshold` and `idx.dead_time_merge_gap` and `idx.min_action_window_duration` exist.

28. user (2026-06-18T07:49:03.583Z)

```
43:    velocity_threshold: float = 0.02
55:    """Typed config for the live WASB shuttle substrate (indexing.wasb).
57:    Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
64:    conda_env: str = "wasb"
65:    weights_path: str = "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
69:    velocity_threshold: float = 0.02
95:    substrate: Literal["wasb", "tracknet"] = "wasb"
97:    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
98:    # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
99:    velocity_threshold: float = 0.02
117:class IndexingConfig(BaseModel):
122:    dead_time_merge_gap: float = 2.0
127:    yolo_velocity_threshold: float = 0.15
131:    min_action_window_duration: float = 2.0
182:    # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff entirely
and
191:    wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
```

29. assistant (2026-06-18T07:49:26.603Z)

All attributes exist. The `SERVED` preset construction is safe.

Now let me dig into the one area I want to confirm is a real defect ? the `compare()` over_seg_guard with rows from `score_intervals`. Let me re-read: `score_intervals` returns both `merge_rate`/`split_rate` AND `split_count`/`merge_count`. The guard's `_cost` with `use_rates=True` reads `merge_rate`/`split_rate` ?, and per-video regression reads `merge_count`/`split_count`/`merge_rate` ?. All present. Good.

Let me now scrutinize the `tolerance_sweep` monotonicity and one final thing: the Hungarian path when `n != m`. `linear_sum_assignment` on a square `size×size` matrix. When `n < m`, rows `n..size-1` are padding rows full of `big`. The solver assigns those padding rows to some columns. The guard `if i < n and j < m and cost[i,j] < big` drops them. Correct. When `n > m`, columns `m..size-1` are padding. Same guard handles it. Fine.

Let me check `eval_stats.paired_delta_ci` for the n=0 case used by `served_regresses` and `promote_if_served_safe`. With empty inputs, `n=0`, `deltas=[]`, `mean_with_ci([])` returns mean 0.0, ci (0,0), so `ci_excludes_zero=False`, `regresses` = `(0.0 < -eps) or (0.0 < -eps)` = False (for eps?0). `served_safe=True`. That means with NO served data, `promote_if_served_safe` treats it as served-safe. Combined with `should_promote_for_user` which would return `promote=False` for n=0 (floor not met). So no false promotion. Fine.

Now let me examine the actual potential bug I want to confirm. Let me look at `_calibrate_and_tune` in experiment.py ? the threshold grid `range(1, 20)` gives t=0.05..0.95, and `best_t` initialized to `variant.threshold`. If all `mean_f1` are 0 (or negative impossible), `best_f1=-1.0` so the first `t=0.05` always wins. That overrides `variant.threshold` even when it's worse. But that's a minor design quirk, not a defect.

Let me look very carefully at `_feature_matrix` when `vc.intervals` is empty but `feature_names` non-empty: `cols = [_feature_column(...)]` each returns `[0.0]*0 = []`. Then `zip(*cols)` over empty lists yields nothing ? `[]`. Fine. And the `else` branch `[[] for _ in vc.intervals]` only when `cols` empty (no feature names). Fine.

Let me check the regression.py guard once more for a real issue: the incommensurable-comparison guard at lines 132-141. If `b_iou` differs, it sets `regressed=True` but STILL proceeds to

compare `s.precision < base_p - tolerance`. That's fine (flag + still report).

Let me look for a real off-by-one or boundary issue in `_points_in` of rally_gate (`start*fps <= p.frame <= end*fps`) ? inclusive both ends, fine.

Let me now examine a subtle issue I want to verify in `over_seg_guard`: the `eps` default is `1e-9` and `net_cost_ok = net_delta <= eps`. Test `test_over_seg_guard_empty_is_trivially_pass` expects `passes=True, n=0`. With n=0: base_net=0, cand_net=0, net_delta=0, net_cost_ok = `0 <= 1e-9` = True, no_merge_rate_regress=True (empty), passes=True. ?

I've now thoroughly reviewed all target files. Let me do a final check on one thing that nags me: the `mean_with_ci` `bootstrap_ci` percentile indexing for very small `n_boot`. And the `_percentile` helper. These are stable.

Let me verify there are no issues with the `compare` in rally_seg_eval passing `merge_rate`/`split_rate` ? let me double-check that `over_seg_guard`'s default `use_rates=True` and the rows actually have those keys by re-reading score_intervals output. Yes, lines 69: `"merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"]`. Confirmed present.

Let me run a quick targeted check: is there a test that actually exercises `compare()` end-to-end with the guard to confirm the keys flow? And check `experiment.py` `_is_fusion_schema` callers to confirm the "moved" function signature is consistent.

30. user (2026-06-18T07:49:28.200Z)

```
backend\eval\ablation.py:57:from backend.eval.experiment import Variant, VideoCandidates,
_is_fusion_schema
backend\eval\ablation.py:394:    ablatable subset cleanly (see ``_is_fusion_schema``)."""
backend\eval\ablation.py:400:        if not _is_fusion_schema(cands):
backend\eval\fusion_audio.py:23:from backend.eval.experiment import _is_fusion_schema
backend\eval\fusion_audio.py:61:    crashed on, so a mixed corpus loads the fusion subset
cleanly (see ``_is_fusion_schema``).
backend\eval\fusion_audio.py:69:        if not _is_fusion_schema(cands):
backend\eval\experiment.py:180:def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) ->
bool:
backend\eval\fusion_compare.py:23:from backend.eval.experiment import _is_fusion_schema
backend\eval\fusion_compare.py:46:    fusion subset cleanly (see ``_is_fusion_schema``)."""
backend\eval\fusion_compare.py:52:        if not _is_fusion_schema(cands):
tests\test_ablation.py:408:        if ablation._is_fusion_schema(cands):
tests\test_experiment.py:18:    _is_fusion_schema,
tests\test_experiment.py:58:def test_is_fusion_schema_distinguishes_lean_and_full():
tests\test_experiment.py:66:    assert _is_fusion_schema(full) is True
tests\test_experiment.py:67:    assert _is_fusion_schema(no_audio) is True
tests\test_experiment.py:68:    assert _is_fusion_schema(lean) is False
tests\test_experiment.py:69:    assert _is_fusion_schema([]) is False
```

31. assistant (2026-06-18T07:49:35.740Z)

Note: `test_ablation.py:408` references `ablation._is_fusion_schema`, and `ablation.py:57` imports it (`from backend.eval.experiment import ... _is_fusion_schema`), so it's re-exported into ablation's namespace. That works. The consolidated function is consistent across all three loaders.

Let me verify the three loaders use the function consistently ? checking the `c["shuttle"]` access that `_is_fusion_schema` is meant to guard. Let me look at one loader to confirm the guard actually prevents the crash it claims to.

32. user (2026-06-18T07:49:36.616Z)

```
40          """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a VideoCandidates
41          (intervals + the 10 feature columns + golden gts). The harness derives labels from
gts
42          itself.
43
```

```
44          Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the
schema the
45          corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus loads
the
46          fusion subset cleanly (see ``_is_fusion_schema``)."""
47          videos: List[experiment.VideoCandidates] = []
48          for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
49              with open(path, encoding="utf-8") as f:
50                  d = json.load(f)
51              cands = d["candidates"]
52              if not _is_fusion_schema(cands):
53                  logger.info("fusion_compare.load_videos: skipping %s ? windowing-only schema
"
54                              "(no shuttle/person cue dicts), not a fusion-feature file",
55                              os.path.basename(path))
56                  continue
57              intervals = [(float(c["start"]), float(c["end"])) for c in cands]
58              features: Dict[str, List[float]] = {}
59              for fn in SHUTTLE_FEATURE_NAMES:
60                  features[fn] = [float(c["shuttle"][fn]) for c in cands]
61              for fn in PERSON_FEATURE_NAMES:
62                  features[fn] = [float(c["person"][fn]) for c in cands]
63              gts = [(float(a), float(b)) for a, b in d["gts"]]
64              videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
65                                                       features=features, gts=gts))
66          return videos
67
68
69      def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
70                  threshold: float = 0.5) -> Dict[str, Any]:
71          shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
72                                            threshold=threshold)
73          fusion = experiment.Variant(name="shuttle_person_fusion",
74                                      feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES),
75                                      threshold=threshold)
76          return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
77
78
79      def format_result(res: Dict[str, Any], iou: float) -> str:
```

33. assistant (2026-06-18T07:49:49.634Z)

The guard correctly prevents the `c["shuttle"]` KeyError. `_is_fusion_schema` checks `all("shuttle" in c and "person" in c for c in candidates)`, and the loader accesses `c["shuttle"][fn]` / `c["person"][fn]`. The guard ensures the top-level keys exist. (It doesn't guard the inner `[fn]` keys, but that's the established contract ? a malformed shuttle dict would be a data-integrity issue beyond this function's scope.)

I've now thoroughly reviewed all the target files plus their dependencies and tests. Let me do a final careful pass on the one thing that could be a genuine subtle defect: the `tolerance_sweep` monotonicity guarantee vs the strict 1:1 matching. Actually, let me reconsider whether there's a real bug in the Hungarian path that the greedy path doesn't have ? specifically a case where minimizing total cost drops a match that maximum-cardinality would keep.

Consider preds `[(0,5)]`, gts `[(0,5),(1.9,6.4)]` with tau_start=2, tau_end=1.5. Pred eligible to gt0 (cost 0) and gt1 (|0-1.9|=1.9?2, |5-6.4|=1.4?1.5, cost 3.3). Hungarian on 2×2: row0=pred costs [0, 3.3], row1=padding [1e6,1e6]. Min total: assign pred?gt0 (0) + pad?gt1 (1e6). Matches: pred?gt0. k=1. Greedy: same. Consistent. No cardinality loss.

The "big penalty dominates" reasoning holds generally: any assignment using more eligible cells saves ?1e6 per cell vs padding, and real eligible costs are bounded by ~3.5, so the optimal-cost assignment is also maximum-eligible-cardinality. The two paths agree on cardinality. No defect.

After a comprehensive read of all eight target files plus their key dependencies (`metrics.py`, `eval_stats.py`, `segmentation_metrics.py`, `rally_gate.py`, `config/models.py`) and the test contracts, I did not find any real, reachable defect. The numeric edge cases (n=0/1, empty inputs, div-by-zero) are all guarded; the schema accesses are guarded; the matching boundaries are correct; the Hungarian/greedy paths agree on cardinality.

34. user (2026-06-18T07:49:50.794Z)

Structured output provided successfully

35. assistant (2026-06-18T07:50:07.168Z)

I completed a thorough adversarial review of the recently-changed `backend/eval/` quality code and found no real, reachable defects. Empty list is my honest answer.

What I verified (read the actual code + matching tests + key dependencies):

- **`tolerance_metrics.py`** ? `match_1to1` guards n=0/m=0 ? returns zeros (no div-by-zero in `precision=k/n`, `recall=k/m`). The Hungarian path (`linear_sum_assignment` over a `big=1e6`-padded square matrix) and the greedy fallback agree on cardinality: any real eligible cost is bounded ~3.5 ? 1e6, so cost-minimization also maximizes eligible-match count, and the `cost[i,j] < big` filter drops forced ineligible assignments. `boundary_error_report` uses `max(preds, key=..., default=None)` and guards `best is not None`. `over_seg_guard` n=0 path returns trivial pass (confirmed by `test_over_seg_guard_empty_is_trivially_pass`); `_aligned` filters `if b["name"] in cmap` (no KeyError); `_cost` reads keys that `score_intervals` actually emits.
- **`rally_seg_eval.py`** ? `evaluate` skips videos with empty gts so `merge_rate = gts_merged/m` has m?1; the rows fed to `over_seg_guard` carry `merge_rate`/`split_rate`/`merge_count`/`split_count` (all present in `score_intervals`). `windows_for` calls `gate_windows(...)` with a matching signature and `g.kept`; never-empty fallback is correct.
- **`served_gate_a.py`** ? `aggregate` `_mean` guards `n==0`; `seg_ratio` via `segment_count_ratio` returns 0.0 for empty gts.
- **`windowing.py`** ? every `IndexingConfig` attribute referenced by `_served_preset_from_defaults`/`served_windowing_from_config` exists in `config/models.py`, so `SERVED` constructs at import without crashing. `_as_mapping` handles dict/typed/None.
- **`experiment.py`** ? the consolidated `_is_fusion_schema` (`all("shuttle" in c and "person" in c ...)`) correctly guards the `c["shuttle"][fn]` access in all three loaders (fusion_compare/fusion_audio/ablation), and `[]` returns False. `_feature_matrix`/`_feature_column` handle empty intervals and short/missing columns.
- **`per_user_gate.py` / `promotion.py`** ? n=0 paths produce honest non-promoting results; `served_regresses` with empty input is not a false "regress"; structural-guard logic matches the docstring.
- **`regression.py`** ? incommensurable-baseline guard flags-and-continues correctly; `gt_hash` normalizes deterministically.

One non-defect note (not reported, pure comment drift): several windowing.py docstrings say a knob "defaults to 0=OFF," but the live `IndexingConfig` defaults are non-zero (`max_window_duration=6.0`, `window_inactivity_end=1.5`). The code reads the real config value, so there's no runtime impact ? only the inline comments are stale.